

文件編號：ICST-C-004

103年資安服務暨專案管理辦公室

安全軟體發展流程指引

(V1.0)

執行單位：財團法人資訊工業策進會

中華民國103年6月

報告摘要

報告名稱	安全軟體發展流程指引
資訊等級	☐機密　☐密　☐內部使用　☒普通
相關撰稿人	蔡宗霖、蔡思玟、陳佳伶、楊惟傑
閱讀對象	☐一般主管　☐資安人員　☒資訊人員　☐一般使用者

內容摘要：

一、目的

本指引針對安全軟體開發生命週期進行深入介紹，介紹傳統開發生命週期與安全軟體生命週期之相似與相異之處，並針對國際 ISO 標準、資安組織與單位制定之方法論與流程進行介紹與分析，並制定適用於安全軟體開發生命週期各階段應遵循之導入事項，提供機關與單位進行安全軟體發展生命週期之推動參考。

二、適用對象

本指引適用之對象為軟體開發人員、系統規劃人員與委外承辦人員。

- 委外承辦人員，著重參考之實務階段為「安全軟體委外開發管理」與「風險評估管理」。委外專案需識別安全需求、確保安全活動被執行並驗證安全需求被滿足，以及將風險管理活動整合至安全軟體發展流程中。
- 系統規劃人員，著重參考之實務階段為「安全軟體需求」、「安全軟體設計」、「安全軟體測試」、「安全軟體部署與維護」。
- 軟體開發人員，著重參考之實務階段為「安全軟體開發」，避免常見軟體漏洞及實作安全控制措施，進行源碼安全檢查，建立與保護安全軟體開發環境。

三、章節架構

第 1 章說明本指引之目的、適用對象及章節架構介紹。

第 2 章第 1、2 節說明介紹「傳統軟體發展流程」、「安全軟體發展流程」，以了解兩者之差異。

第 2 章第 3 節介紹「軟體安全發展相關國際標準」，如 ISO/IEC 27001:2013、ISO/IEC 27034:2011、NIST SP 800-64，作為本指引參考文獻。

第 2 章第 4 節「導入安全軟體發展流程之管理重點」，說明組織若要導入安全軟體發展流程，於管理制度面需要注意的地方，包含「策略與評量」、「政策與法規遵循」與「教育訓練」。

第 3 章開始，就安全軟體發展流程中，各階段的實務活動進行說明。

第 3 章第 1 節「安全軟體需求」，說明介紹軟體安全需求類型，進行需求萃取方式，以及後續追蹤的需求追溯矩陣。

第 3 章第 2 節「安全軟體設計」，說明進行設計時應遵循的安全軟體設計法則，以及安全軟體設計風險分析方法，包含「威脅建模」、「架構風險分析」等。於設計階段，進行安全設計審查，確保設計符合安全需求。

第 3 章第 3 節「安全軟體開發」，說明開發過程中，應避免的常見軟體漏洞及因應相關漏洞之控制措施。程式設計師撰寫程式碼後，應進行程式碼安全檢查，並注意其開發環境的安全性。

第 3 章第 4 節「安全軟體測試」，說明軟體測試階段針對安全性進行之源碼檢測與滲透測試，以及此階段需驗證安全需求的實作結果是否符合預期要求。

第 3 章第 5 節「安全軟體部署與維運」，說明介紹軟體上線需進行部署環境的強化，後續軟體變更的管理，軟體維運中的監控量測，以及發生資安

事故的處理。

第 3 章第 6 節「安全軟體委外開發管理」，說明軟體委外各階段中，如何使用本指引介紹之實務活動內容輔助進行委外管理。

第 4 章「結論」，總結各章節內容。

第 5 章「參考文獻」，詳列本指引參考的文件資料。

第 6 章「附件」，詳列本指引的相關附件內容。

關鍵詞	安全軟體發展生命週期、安全軟體開發、安全軟體設計

目　次

1. 前言 1
1.1. 目的 1
1.2. 適用對象 1
1.3. 章節架構 1
2. 安全軟體發展流程介紹 4
2.1. 傳統軟體發展流程 4
2.2. 安全軟體發展流程 11
2.3. 安全軟體發展相關標準介紹 28
2.4. 導入安全軟體發展流程之管理重點 42
3. 安全軟體發展流程階段實務 50
3.1. 安全軟體需求 50
3.2. 安全軟體設計 59
3.3. 安全軟體開發 85
3.4. 安全軟體測試 95
3.5. 安全軟體部署與維運 109
3.6. 安全軟體委外開發管理 118
4. 結論 123
5. 參考文獻 124
6. 附件 127
附件 1　軟體安全問卷範例 127
附件 2　需求追溯矩陣範例 127
附件 3　源碼安全查檢表範例 127

圖 目 次

圖 1 微軟 SDL 12
圖 2 Organization Normative Framework 34
圖 3 Application Security Management Process 關係圖 35
圖 4 ISO/IEC 27034:2011 與 Open SAMM 對應關係 36
圖 5 NIST SP 800-64 系統發展生命週期示意圖 38
圖 6 軟體安全需求類型 51
圖 7 誤用案例範例 56
圖 8 Data Flow Diagram 範例 67
圖 9 攻擊樹狀圖範例 71
圖 10 架構風險分析活動流程示意圖 76
圖 11 可存取性影響攻擊面 81
圖 12 安全設計審查流程 82
圖 13 CAPTCHA 87
圖 14 防範物件參考不安全之範例 91

表 目 次

表 1　CLASP 實務與活動對應表 24
表 2　ISO/IEC 27001:2013 附錄 A.14 控制措施與本指引章節關係對應表 30
表 3　軟體的有形資產 63
表 4　資料流程圖的組成元素範例 66
表 5　STRIDE 68
表 6　DFD 元素對應的威脅類型 69
表 7　威脅描述範例 69
表 8　威脅評分表範例 72
表 9　威脅對應評分表範例 73
表 10　威脅與相對應控制措施範例 74
表 11　軟體安全特點檢視結果範例 83

1. 前言

1.1.目的

本指引針對安全軟體開發生命週期進行深入介紹，介紹傳統開發生命週期與安全軟體生命週期之相似與相異之處，並針對國際 ISO 標準、資安組織與單位制定之方法論與流程進行介紹與分析，並制定適用於安全軟體開發生命週期各階段應遵循之導入事項，提供機關與單位進行安全軟體發展生命週期之推動參考。

1.2.適用對象

本指引適用之對象為軟體開發人員、系統規劃人員與委外承辦人員。

- 委外承辦人員，著重參考之實務階段為「安全軟體委外開發管理」與「風險評估管理」。委外專案需識別安全需求、確保安全活動被執行並驗證安全需求被滿足，以及將風險管理活動整合至安全軟體發展流程中。
- 系統規劃人員，著重參考之實務階段為「安全軟體需求」、「安全軟體設計」、「安全軟體測試」、「安全軟體部署與維護」。
- 軟體開發人員，著重參考之實務階段為「安全軟體開發」，避免常見軟體漏洞及實作安全控制措施，進行源碼安全檢查，建立與保護安全軟體開發環境。

1.3.章節架構

第 1 章說明本指引之目的、適用對象及章節架構介紹。

第 2 章第 1、2 節說明介紹「傳統軟體發展流程」、「安全軟體發展流程」，以了解兩者之差異。

本文件之智慧財產權屬行政院資通安全辦公室所有。

第 2 章第 3 節介紹「軟體安全發展相關國際標準」,如 ISO/IEC 27001:2013、ISO/IEC 27034:2011、NIST SP 800-64，作為本指引參考文獻。

第 2 章第 4 節「導入安全軟體發展流程之管理重點」，說明組織若要導入安全軟體發展流程，於管理制度面需要注意的地方，包含「策略與評量」、「政策與法規遵循」與「教育訓練」。

第 3 章開始，就安全軟體發展流程中，各階段的實務活動進行說明。

第 3 章第 1 節「安全軟體需求」，說明介紹軟體安全需求類型，進行需求萃取方式，以及後續追蹤的需求追溯矩陣。

第 3 章第 2 節「安全軟體設計」，說明進行設計時應遵循的安全軟體設計法則，以及安全軟體設計風險分析方法，包含「威脅建模」、「架構風險分析」等。於設計階段，進行安全設計審查，確保設計符合安全需求。

第 3 章第 3 節「安全軟體開發」，說明開發過程中，應避免的常見軟體漏洞及因應相關漏洞之控制措施。程式設計師撰寫程式碼後，應進行程式碼安全檢查，並注意其開發環境的安全性。

第 3 章第 4 節「安全軟體測試」，說明軟體測試階段針對安全性進行之源碼檢測與滲透測試，以及此階段需驗證安全需求的實作結果是否符合預期要求。

第 3 章第 5 節「安全軟體部署與維運」，說明介紹軟體上線需進行部署環境的強化，後續軟體變更的管理，軟體維運中的監控量測，以及發生資安事故的處理。

第 3 章第 6 節「安全軟體委外開發管理」，說明軟體委外各階段中，如何使用本指引介紹之實務活動內容輔助進行委外管理。

本文件之智慧財產權屬行政院資通安全辦公室所有。

第 4 章「結論」，總結各章節內容。

第 5 章「參考文獻」，詳列本指引參考的文件資料。

第 6 章「附件」，詳列本指引的相關附件內容。

本文件之智慧財產權屬行政院資通安全辦公室所有。

2. 安全軟體發展流程介紹

2.1.傳統軟體發展流程

傳統軟體發展流程遵循軟體發展生命週期的各階段流程。軟體發展生命週期(Software Development Life Cycle, SDLC)意指發展一套系統的順序，用以開發完善的資訊系統[1]。一般可分為如下幾個階段：

●研究規劃

此階段目的為找出問題及可被資訊系統滿足的機會，其任務包含：

– 界定目標、範疇。

– 發現系統需求及對問題之了解。

– 明確辨認問題之性質及範圍：包括人、事、組織、經費、系統架構。

– 實施單位之權責/權限。

– 規劃單位/委託單位/業務單位之互動情形。

– 使用者需求。

– 情境分析。

– 研擬方案。

– 可行性研究。

– 成本效益評估。

– 向管理單位提出建議：包括人、事、組織、經費、時程、相關技術等。

●需求分析

本文件之智慧財產權屬行政院資通安全辦公室所有。

此階段著重需求定義，以符合業務內容及使用者需求為目的。需求分析指的是在建立一個新的或改變一個現存的系統或產品時，確定新系統的目的、範圍、定義、功能及所要做的所有工作。需求分析是軟體工程中的一個關鍵過程。在這個過程中，分析師確定顧客的需要。只有在確定這些需要後，才能夠分析和尋求解決方法，其主要技術包含[2]：

– 採訪具有重要訊息或了解業務內容的人

採訪具有重要訊息的人是需求分析中一個必不可少的過程。在一個大型的系統中，可能有許多人必須被訪談，這表示必須花費許多金錢和時間成本，但最重要的是這個過程，最可能顯示現有的業務流程與新系統中的業務流程之間的差別。不同的使用者角色有可能有不同的或甚至相對的需求，在這種情況下，分析師必須協調各方的需要。

– 需求工作會議

基於上述原因，若系統非常複雜時，需求分析最常用的方法是召開需求工作會議，於需求工作會議上，分析師和具有重要訊息的人一起分析系統的需要和探討解決方案。

– 將需求列成契約式的檔案

最常見的記錄需求方式，是將顧客需求列入一個契約式的文件中。

– 雛型(Prototype)系統

從 1980 年代中開始，越來越多的人將雛型系統看作是解決需求分析困難的辦法。雛型系統模擬最終軟體的螢幕顯示，這樣使用者可以看到最終軟體將是什麼模樣，而在這些螢幕顯示的背後，實際的程式邏輯尚未被開發。這樣顧客可以在系統還沒有建立之前就做出流程及畫面設計的

本文件之智慧財產權屬行政院資通安全辦公室所有。

決定。引入雛型系統往往提高顧客與開發者之間的訊息交換，透過此作法，螢幕顯示在後來的階段中往往較少被改變，因此可以大大地降低成本。

– 使用案例(Use Case)

使用案例是一種記錄新功能或功能異動需求的技術。每個使用案例包含系統在作業時與使用者或與其他系統之間交換資訊的情境。使用案例應避免使用術語，而儘量使用顧客、使用者或他們專業領域的語言。

在 1990 年代中，使用案例很快地成為記錄需求分析的最主要的方式，在物件導向的程式設計中，使用案例普及性非常高，但應用上並不侷限於物件導向之設計。

●系統設計

系統設計是一個解決問題和規劃軟體解決方案的過程，在確定軟體目的和需求後，軟體開發者需提出解決方案的計畫，包括軟體架構、邏輯模型、演算法及底層的軟體組件組成方式等，可區分為結構化分析設計及物件導向分析設計方式：

– 結構化分析設計

結構化系統分析及設計方法中，最主要為以下 3 項技術：

➢邏輯資料建模(Logical Data Modeling)

針對要建立系統需要的資料，加以識別、建模並建立文件的過程。資料可分為實體(Entity)及關係，實體是指商業上需要紀錄資訊的某種事物，而關係是指實體之間的交互關係。

本文件之智慧財產權屬行政院資通安全辦公室所有。

➢資料流建模(Data Flow Modeling)

此為資訊系統中資料轉換的方式,加以識別、建模並建立文件的過程。資料流建模研究程序(由一種資料轉換為另一種資料的活動)、資料儲存、外部實體(會傳送資料給系統，或由系統接收資料的事物)及資料流(資料流動的路徑)。

➢實體表現建模(Entity Behavior Modeling)

針對要影響實體的事件，以及事件之間的先後順序，加以識別、建模並建立文件的過程。對傳統的軟體工程來說，採用每個使用案例描寫系統的一個特點。

– 物件導向分析設計

採用統一塑模語言(Unified Modeling Language, UML)進行系統分析設計。UML 是依據「物件導向式方法」所發展出來的一種描述資訊系統的圖式語言，並已成為「物件管理組織」(Object Management Group)以及國際標準組織(ISO)所認可的標準。共使用以下 9 種圖形進行建模：

➢使用案例圖

表示系統的使用個案和行為者之間互動的關係。

➢類別圖

系統存在之類別以及類別間的邏輯關係，該圖是 UML 模式圖的核心。

➢物件圖

用以描述系統於某一時間點的靜態物件結構。

本文件之智慧財產權屬行政院資通安全辦公室所有。

➢循序圖

用以描述系統運作時，物件間的互動行為且著重以時間為主軸的處理程序。

➢合作圖

該圖能同時展現物件間的資料流程、控制流程與訊息傳遞的活動總流程。

➢狀態圖

用以表達物件在其生命週期中的狀態變化。狀態圖是以微觀物件為主，細分物件所發生的各項事件，並表達物件生命週期之狀態轉變及活動結果。

➢活動圖

描述一群循序與同步的活動，一個活動狀態表示一個工作流程步驟或一個運算的執行活動。

➢元件圖

用以說明系統設計過程，各類別與物件的配置及敘述軟體元件間的組織架構和相依關係。

➢部署圖

用來說明系統各處理元件的配置與關聯。

●系統開發

本文件之智慧財產權屬行政院資通安全辦公室所有。

系統開發為解決特定問題程式的過程，為軟體建構活動中的重要階段，程式設計往往以某種程式語言為工具，實現相關的軟體功能。

●系統測試

系統測試為確保軟體的正確性、完整性、安全性和品質的過程，常見測試類型包含：

– 功能測試

功能測試的重點為驗證主要需求是否直接可以反應到使用案例的設計上。測試的目標在於驗證資料正確的接受，處理與並傳回，此過程須符合本單位的商業邏輯或規則。

– 使用者介面測試

使用者介面測試主要為驗證使用者與系統之間的互動，其目標為確保使用者介面提供使用者正確的存取系統功能與瀏覽。

– 效能測試

效能測試衡量回應時間、交易完成的速度與其他跟時間相關的需求，效能測試的目標是驗證有關效能的需求被達成。

➢負載測試

負載測試測量系統在不同程度的負載下能夠繼續正確運作的能力。

➢壓力測試

壓力測試要求進行超過規定效能指標的測試，亦可視為負載測試的一種，即超過系統負載的高負載測試。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– 設定測試(Configuration Testing)

驗證軟體在不同的軟硬體設定下的操作是否正確，多數的情況下客戶端的軟硬體設定不盡相同。設定測試需要找出與確認怎樣的設定下，系統可以正確的運作。

– 安裝(Deployment)測試

安裝測試主要為驗證軟體可以依照標準的程序步驟完成安裝部署的工作。

●系統部署

泛指使系統或軟體可上線正式運作的所有必要活動。包含：

– 硬體建置。

– 軟體安裝。

– 系統配置設定。

– 上線測試。

– 教育訓練。

●系統維護

系統維護包含系統服務之運作維持和更新，確保穩定的服務品質。對於系統或軟體產品在交付上線後，為修正錯誤、增進效能或其他品質所進行的修改。相關流程如下：

– 實行軟體維護的準備流程

例如建立維護計畫，準備處理問題的方式以及軟硬體設定管理。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– 問題及變更分析流程

如何分析、確認、調查問題並提出解決方案，記錄需求及建議的解決方案，以及獲取所有的授權以進行後續異動。

– 發展實作該項異動的流程

例如實際撰寫修正程式碼，並上線至運作環境。

– 變更接受流程

透過與需求提出者確認變更的作業確保異動滿足需求。

– 系統轉移流程

此流程並非日常維護作業，而是當系統有轉移平台或變更開發技術時可能會發生。

2.2.安全軟體發展流程

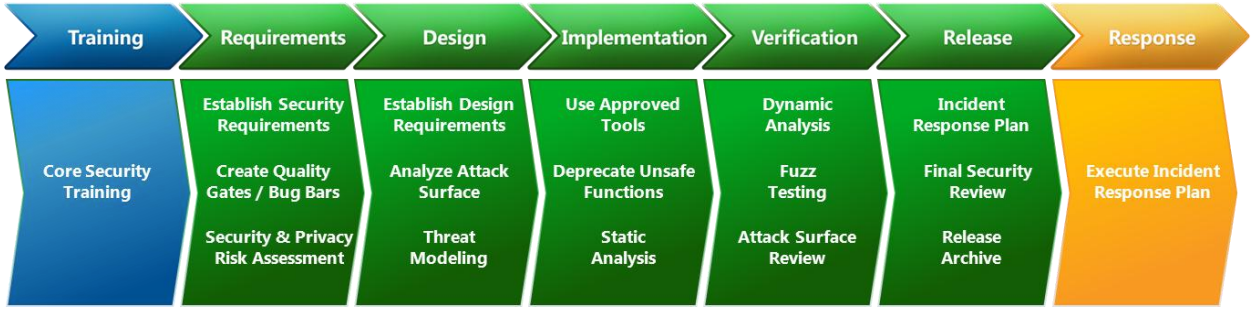
傳統軟體發展流程為功能性導向，期望在最短的時間，完成系統的開發與上線，缺乏安全性考量的設計，依賴外部的安全機制，例如防火牆、入侵偵測系統等，然而這些機制並不能解決所有資安問題，尤其是系統設計上、邏輯層次的安全瑕疵及缺點。

安全軟體發展流程(Secure Software Development Life Cycle, SSDLC)則考量系統功能性的同時，導入安全性的思維，於系統開發流程中，進行各項必要的安全活動，雖拉長了各階段的時程，卻降低了系統後續維護的成本，以及遭受到攻擊損失的可能性。以下將介紹兩個著名的安全軟體發展流程，分別為微軟的「安全發展生命週期」(Security Development Lifecycle, SDL)與 OWASP 組織的 CLASP。

本文件之智慧財產權屬行政院資通安全辦公室所有。

2.2.1. 微軟-「安全發展生命週期」SDL

2000 到 2001 年時，由於網際網路的發展，頻繁發生大量繁殖的病毒、蠕蟲等針對微軟作業系統的安全威脅。Bill Gates 在 2002 年對微軟公司員工發出的備忘錄中，提到了高信賴度運算(Trustworthy Computing, TwC)的概念。重要的成果就是 2004 年推出的 Windows XP Service Pack 2，例如開始內建個人防火牆、增加了資料執行防止(Data Execution Prevention, DEP)、強化對 IE 瀏覽器外掛的管理、自動啟動系統更新機制等。後續微軟也逐漸意識到自身對資安威脅反應力不足的問題，必須找到正確的方法來從根本解決。故微軟於 2004 年開始，將自行發展的「安全發展生命週期」作為全公司實行的政策作法，該作法區分 7 階段，16 項必要實務活動，3 項選擇性實務活動[3]：



資料來源：[3]

圖1　微軟 SDL

●準備階段：安全教育訓練

– 所有軟體開發團隊中的成員必須接受合適的教育訓練，以具備資安意識以及對最近的資安威脅保持警覺。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– 與開發流程直接相關的技術團隊(包含開發人員、測試人員、專案管理人員)，每年至少 1 次接受單獨的訓練課程。

– 基礎的軟體安全訓練應該包含以下範圍：

➢安全軟體設計，包含如何減少攻擊面(Attack Surface)、縱深防禦、最小權限法則與預設安全性。

➢安全的程式開發，包含緩衝區溢位、SQL 資料隱碼攻擊、跨站腳本攻擊與避免脆弱的密碼學應用。

➢安全的軟體測試，包含安全測試與功能性測試的差異、風險評鑑與安全測試方法。

➢隱私保護，包含隱私資料類型、隱私保護之設計實務、隱私保護之開發實務與隱私保護之測試實務。

●第 1 階段：需求管理

– 考量對於安全性的需求

預先考量系統安全與隱私是安全軟體發展的基礎觀念。定義安全需求的最佳時機是在系統初始規劃階段，及早定義安全需求使得開發團隊可以識別關鍵的查核點與產出，以及使得整合安全與隱私至系統的工作對原先的計畫與時程影響最小化。

– 使用品質關卡(Quality Gates)或 Bug Bars

本項目主要為建立對於軟體安全及隱私品質最低的接受水準。於專案開始階段定義規範水準，有助了解與安全議題相關的風險並幫助團隊識別及修復相關安全議題的弱點。專案團隊需溝通每個發展流程階段的品質

本文件之智慧財產權屬行政院資通安全辦公室所有。

關卡，然後經過團隊內安全專家核可，對專案特定要求進行澄清或增加更嚴謹安全需求。專案團隊需列舉證據合乎品質關卡規範，以完成最終安全審查(Final Security Review, FSR)。

Bug Bars 則是一種應用於整個軟體發展專案過程的品質關卡。例如在軟體版本釋出時，不可以有任何嚴重或重要性高的弱點尚未修補，Bug Bars 一旦被設定就不應該有例外。

– 進行安全性與隱私風險評鑑

安全風險評鑑 (Security Risk Assessment, SRA) 和隱私風險評鑑 (Privacy Risk Assessment, PRA)是必須的過程，用於確定軟體中需要深入評析的功能環節。這些評鑑必須包括以下資訊：

- (安全)專案的哪些部分在發布前需要威脅模型？
- (安全)專案的哪些部分在發布前需要進行安全設計審查？
- (安全)專案的哪些部分(如果有)需要由不屬於專案團隊且雙方認可的小組進行滲透測試？
- (安全)是否存在安全顧問認為有必要增加的測試或分析要求以緩解安全風險？
- (安全)模糊測試要求的具體範圍是什麼？
- (隱私)隱私影響評級如何？應基於以下準則回答此問題：
  - ◆P1-高隱私風險：功能、產品或服務將儲存或傳輸個資 (Personal Identification Information, PII)，更改設置或檔案類型關聯，或是安裝軟體。

本文件之智慧財產權屬行政院資通安全辦公室所有。

◆P2-中等隱私風險：功能、產品或服務中影響隱私的唯一行為是用戶啟動的一次性匿名資料傳輸(例如，軟體在使用者按一下連結後轉到外部網站)。

◆P3-低隱私風險：功能、產品或服務中不存在影響隱私的行為。不傳輸匿名或個人資料，不在電腦上儲存個資，不代表使用者更改設置，並且不安裝軟體。

●第 2 階段：設計管理

– 檢視設計需求

影響軟體專案設計層面可信賴度的最佳時間點是早期的軟體發展生命週期。於設計階段考量安全與隱私為必須之步驟，因為在此階段消除安全性與隱私性議題的成本將遠低於後續階段，專案團隊應避免在軟體開發的後期才置入安全性與隱私性的功能。

設計需求包含一些必須活動，如創建安全和隱私設計規格、規格審查以及最低加密設計規格要求。

設計規格應描述使用者會直接存取的安全或隱私功能，例如需要使用者身分驗證才能存取特定資料，或於使用高風險隱私功能前需要使用者同意的特定功能。

此外，所有設計規格都應描述如何安全地實作，針對應用程式的功能規格去驗證設計規格是個好作法。功能規格應：

➢準確完整地描述特性或功能的預期用途。

➢描述如何以安全的方式部署特性或功能。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– 設法減少受到攻擊的面

減少攻擊面與威脅建模緊密相關，但解決問題的角度稍有不同。減少攻擊面透過減少攻擊者刺探弱點的機會以降低風險；減少攻擊面強調直接切斷或限制存取系統不必要服務，應用最小權限法則與縱深防禦的概念。

– 威脅建模(Threat Modeling)

威脅建模用於存在重大安全風險的環境之中。他允許開發團隊在計畫的運行環境中，以結構化的方式去考量、記錄與討論設計的安全影響。威脅建模也允許在元件或者應用程式的層級考量安全性議題。威脅建模是團隊進行之活動，進行過程中參與角色包含專案經理、開發者與測試人員，為軟體設計階段主要的風險分析任務。請參考 3.2.2.1 節。

●第 3 階段：實作階段

– 採用通過檢驗的工具

開發團隊應該定義並公布一份被組織驗證使用之工具名單及相關的安全檢查紀錄，此名單應被團隊中的安全顧問核可。一般來說，開發團隊應使用最新版本的工具以確保受到較完善的安全防護措施。

– 拒用不安全的函式

許多常用的函式或 API 面臨現今的威脅環境，已經是不安全的。專案團隊應該分析專案使用的所有函式及 API 並禁用那些被判斷為不安全的項目。一旦不安全的項目名單被決定，專案團隊應該使用源碼檢測工具等方式來檢查是否使用這些不安全的項目，並以較安全的替代方案取代。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– 執行源碼的靜態分析

專案團隊應對源碼實施靜態分析，源碼靜態分析可以幫助確保安全源碼撰寫規範的符合性。安全團隊和安全顧問應了解靜態分析工具的優點和缺點，並根據需要為靜態分析工具輔以其他工具或人工檢視。

●第 4 階段：進行驗證程序

– 執行源碼的動態分析

為確保應用程式的安全功能如設計階段預期的正常運作，應用程式執行時期的驗證是必要的。此一任務應使用特定工具監視系統行為，包含記憶體使用、使用者權限、以及其他重要的安全問題。

– 進行模糊測試

模糊測試是一種專門形式的動態分析，透過故意向應用程式輸入格式不正確或亂數資料引發程式功能故障。模糊測試策略的制定以應用程式的預期用途，以及應用程式的功能和設計規範為基礎。安全顧問可能要求進行額外的模糊測試或擴大模糊測試的範圍和持續時間。

– 重新審視威脅模型與攻擊面

應用程式經常會嚴重偏離軟體發展專案需求，以及設計階段所制定的功能和設計規範。因此，在應用程式完成程式開發後重新評估其威脅模型和攻擊面是非常重要的。此評估可確保對系統設計或實作方面所做的全部更動被考量，確保因這些更動而形成的攻擊威脅都被評估與消除。

●第 5 階段：發布

– 準備事故應變計畫

本文件之智慧財產權屬行政院資通安全辦公室所有。

每個因應需求進行發布的軟體，都應該包含事故應變計畫。在發布時沒有包含任何已知弱點的軟體也會因為時間的推展而產生新的安全威脅。事故應變計畫應包含：

- 定義持續維運團隊負責處理安全事故。如果團隊太小無法有獨立的維運人員，則建立緊急應變計畫，以定義安全事故時，工程、行銷、通訊及管理人員的第一線聯絡窗口。
- 具有授權的線上聯絡窗口，可能為 7x24 小時值勤。
- 針對從組織中其他單位取得的源碼之安全服務計畫。
- 針對從第三方協力廠商取得的源碼之安全服務計畫，包括檔案名、版本、源碼、協力廠商聯繫資訊以及要進行異動的契約許可。

– 最終安全審查(Final Security Review, FSR)

最終安全審查是對所有在軟體發布前，所進行安全活動的詳細檢查。最終安全審查由安全顧問在開發團隊及負責安全及隱私團隊領導者協助下進行。最終安全審查並非滲透與修補的活動，也不是一個補行先前遺漏或忘記進行的安全活動的行為。最終安全審查通常包含檢查威脅模型中提出的消除風險對策是否完整執行、自動化分析工具的產出，以及目前效能水準與先前定義品質關卡或 Bug Bars 的比較，最終安全審查的結果是以下三者之一：

- 通過 FSR：

  所有被識別的安全與隱私問題都被修正或消除。

- 通過 FSR，但有異常狀況

本文件之智慧財產權屬行政院資通安全辦公室所有。

所有被識別的安全與隱私問題都被修正或消除，產生的異常狀況也被妥適的解決。無法被處理的問題被記錄並且會在下一次發布前修正完成。

➢需上呈問題的 FSR：

如果團隊無法滿足所有需求並且無法達到可接受的水準，安全顧問無法核可的情況下，軟體專案將不能發布。團隊必須達成原先定義的需求或者將 FSR 結果上呈管理階層進行決策。

– 正式發布、歸檔

被指派負責發布事宜的安全顧問必須證明(使用 FSR 和其他資料)專案團隊已滿足安全要求。同樣，對於至少有一個元件具有 P1 隱私影響等級的產品，專案的隱私顧問必須先證明專案團隊滿足隱私要求，然後才能交付軟體。此外，須對所有相關資訊和資料進行存檔，以便可以對軟體進行發布後維護。

這些資訊和資料包括所有規格、源碼、二進位檔案、專用符號、威脅模型、文件、緊急應變計畫、任何協力廠商軟體的軟硬體授權和服務條款，以及執行發布後維護任務所需的任何其他資料。

●三項選擇性實務活動

– 人工源碼檢查

人工源碼檢查是微軟 SDL 中選擇性的任務，通常由對於軟體安全具有高度技術的個人或者由安全顧問執行。雖然自動化分析工具可以找出許多弱點，但並不是完美無缺。故人工源碼檢查通常聚焦在軟體中重要性

本文件之智慧財產權屬行政院資通安全辦公室所有。

高的元件，如處理或儲存可識別個人資訊的地方；也常被用在驗證重要的安全功能，如密碼學的實作。

– 滲透測試

滲透測試是一種透過具有專業技能的安全專家，模擬駭客行為的安全分析。滲透測試的目的是發掘可能的弱點，包含來自源碼撰寫錯誤、系統設定錯誤或其他部署操作的脆弱點。滲透測試通常與自動及人工源碼檢查一起執行，以提供比平常更進階的分析結果。

– 相類似軟體的弱點分析

在網際網路上可以找到許多聲譽良好的有關軟體漏洞的資訊來源。在某些情況下，透過對類似軟體中發現的漏洞進行分析，可以為所開發軟體中的潛在設計或實作問題獲得一些啟迪。

2.2.2. OWASP - Comprehensive, Lightweight Application Security Process

OWASP(The Open Web Application Security Project)是一個全球性、非營利的組織，專注在增進軟體的安全。CLASP(Comprehensive, Lightweight Application Security Process)是 OWASP 組織中的一個計畫，是應用安全性到組織內軟體開發流程的方法。

CLASP 流程透過 5 個高階、稱作「CLASP 視角」的概念進行呈現。這些視角再被細分為多個包含程序元件的活動。透過「視角」>「活動」>「流程元件」這樣由上而下的組織方式，了解各元件是如何互動以及如何應用到特定的軟體發展流程中。CLASP 包含以下五個視角[4]：

●概念視角

本文件之智慧財產權屬行政院資通安全辦公室所有。

這個視角提供高階對 CLASP 的簡單介紹，例如 5 個視角的互動、7 個 CLASP 最佳實務活動、CLASP 與組織安全政策之關聯以及應用 CLASP 流程元件的順序。

●角色視角

這個視角提供以角色為基礎，說明 CLASP 流程的活動與團隊角色的對應關係。

●活動評鑑視角

這個視角幫助專案經理評鑑 24 個「CLASP 活動」的適用性，並選擇一個子集合在專案中使用。CLASP 也對新起始的專案以及舊專案兩種情況提供了兩個範例，以示範如何選擇適當的活動。

●活動實作視角

此視角包含了 24 個安全相關且可以被整合到軟體開發流程的活動。於活動評鑑視角階段評估後並接受的活動在此階段轉化為軟體實作。

●漏洞視角

此視角包含 104 種被 CLASP 識別的、基於軟體源碼基礎的安全漏洞類型。CLASP 將此 104 種安全漏洞區分成為五個大類：

– 一般邏輯錯誤(General Logic Errors)。

– 範圍與型態錯誤(Range and Type Errors)。

– 環境問題(Environmental Problems)

– 同步與時間錯誤(Synchronization & Timing Errors)。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– 協定錯誤(Protocol Errors)。

CLASP 定義以下 7 個最佳實務(Best Practices)：

●安全意識宣導方案

基礎的安全觀念與技術對於組織內的軟體開發者或相關人員可能是陌生的。因此，在一開始對相關人員進行教育訓練是相當迫切的。專案管理者作為軟體專案發展的驅動者，透過訓練宣示軟體安全作為重要的專案目標，是相當重要的。透過聘僱外部專家作可協助安全意識的教育訓練之進行，確保軟體發展流程中其他階段活動有效率產出安全軟體。

●施行應用程式評鑑

雖然軟體安全無法僅透過測試而建立，軟體測試與評鑑仍然是整體安全策略的重要元素之一。安全評鑑(特別是自動化的測試)可以發現安全問題，找出於實際運作環境的安全風險，亦可作為縱深防禦機制，協助找到設計、規格或實作上的錯誤。測試和評鑑功能通常由測試分析員或品保人員擔任，但其作用可以跨足整個軟體生命週期，不限於傳統流程中實作完成後的測試階段。

●捕捉安全軟體需求

確保安全需求相對於其他必須的系統功能相對重要。一般分析師或專案經理以軟體支援組織業務為重，於定義需求時很容易著重在功能需求而忽略安全考量，故將軟體安全需求作為發展階段明確的部分極為重要。相關須被考量的因素有：

– 對於軟體可能被誤用及攻擊方式的了解。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– 軟體會存取或提供的資訊資產，其相對於組織面臨的風險、法規及對商譽的可能影響，是否具有適當程度的保護措施。

– 軟體架構及可能的攻擊者。

– 可行的控制措施及其花費與效用。

●實作安全開發實務

軟體基本的安全目標為創造及維護可重用的程式碼，強化組織中軟體的基本安全與提供可重複使用性。此目標可透過組織整體的軟體發展生命週期中實施安全開發實務來達成。安全開發實務提供明確定義，以角色為基礎的活動。如協助專案團隊應用軟體安全法則在設計階段、整合動態靜態安全分析到程式碼管理流程。安全開發實務亦提供程式碼撰寫規範，幫助專案團隊系統化避免及排除程式碼中的安全漏洞。

●建置漏洞修補程序

定義如何鑑別、評鑑、設定優先權及漏洞修補的步驟，對於提升軟體安全相當重要。漏洞修補程序將會透過事先定義好處理漏洞的角色、職責及程序，來提升識別漏洞後的回應速度及最小化風險程度。漏洞修補程序通常經由來自第三方或自行進行的軟體安全評鑑產生。

●定義監測安全量測指標

軟體開發專案若無法量測便無法管理。實施一個有效監督品質的量測是困難的工作，但是量測卻是軟體安全品質的基本元素。量測目前軟體的安全情勢是相當重要的，有助於把主要的方向著重在最嚴重的漏洞上，並顯示組織在投入資源後對於安全程度的產出表現。

●發布作業安全指引

本文件之智慧財產權屬行政院資通安全辦公室所有。

軟體安全的工作並非在開發完成並部署上線到正式環境後結束。要使大部分的網路或作業安全的措施有效，需要持續監看及管理上線系統的安全性。日常作業程序中，需要建議及指引該軟體所建立的安全措施如何被正確的使用，以確保其目的確實達成。

CLASP 的 7 個實務包含 24 個安全活動，請詳見表 1。

表1　CLASP 實務與活動對應表

CLASP 實務	CLASP 活動	執行角色
安全意識宣導方案	安全意識宣導方案	專案經理
施行應用程式評鑑	施行軟體需求及設計的安全分析(威脅建模)	安全稽核員
	施行源碼層次的安全檢視	負責人：安全稽核員 貢獻人：實作者、設計者
	識別、實作、執行安全測試	測試分析員
	驗證資源的安全特性	測試者
	研究評鑑技術解決方案的安全程度	負責人：設計者 貢獻人：元件廠商
捕捉安全軟體需求	識別全面的安全政策	需求制定者
	識別資源及信任邊界	負責人：架構師 貢獻人：需求制定者
	識別角色及資源能力	負責人：架構師 貢獻人：需求制定者
	指定作業環境	負責人：需求制定者

本文件之智慧財產權屬行政院資通安全辦公室所有。

CLASP 實務	CLASP 活動	執行角色
		貢獻人：架構師
	詳細的誤用案例	負責人：需求制定者 貢獻人：專案關係人
	識別攻擊面	設計者
	將安全相關需求以文件記錄	負責人：需求制定者 貢獻人：架構師
實作安全開發實務	應用安全設計法則	設計者
	註解安全性質的設計	設計者
	闡述並實施資源政策及安全技術	實作者
	實作使用者介面的限制條件	實作者
	整合安全分析至源碼管理流程	整合者
	執行源碼簽署	整合者
建置漏洞修補程序	管理安全議題揭露流程	負責人：專案經理 貢獻人：設計者
	處理回報的安全議題	負責人：設計者 貢獻人：錯誤回報者
定義監測安全評量	監測安全評量	專案經理
發布作業安全指引	指定資料庫安全設定	資料庫設計者
	建置作業安全指引	負責人：整合者 貢獻人：設計者、架構

本文件之智慧財產權屬行政院資通安全辦公室所有。

CLASP 實務	CLASP 活動	執行角色
		師、實作者

資料來源：本指引整理

CLASP 定義流程中參與的執行角色，茲說明其主要職責如下：

●專案經理(Project Manager)

專案經理首要任務為促進團隊安全意識，團隊成員需要對軟體安全策略有基本的了解，並對團隊成員進行教育訓練。另外，專案經理應該促進軟體專案團隊以外人員的安全意識，組織內其他人員需要知道軟體安全對於組織商業上的影響。另一個專案經理主要的職責是監測狀況，包含定義相關的評量標準及應用其標準。

●需求制定者(Requirements Specifier)

需求制定者主要在於提供商業需求的明細內容，特別是與安全相關時，架構師會一同參與考量。對大部分的組織來說，在安全需求上，這兩個角色應該是緊密合作的。當基本的架構與需求被決定，需求制定者應檢視架構中所使用的資源並決定這些資源需要何種保護。若組織已經事先定義資源所需遵循的防護水準，如此安全需求可成為跨專案應用之共通資源。若專案團隊採用使用案例描述需求，需求制定者可以協助發展誤用案例，其顯示了專案關係人對於系統設計的主要安全考量。

●架構者(Architect)

– 了解專案採用的相關技術對安全的影響，以避免引入明顯的安全錯誤。

– 列舉軟體中所有使用的資源，其層次越深層越詳盡越好。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– 進一步支援建置安全需求，定義系統中哪個角色會使用到何種資源。

– 定義對於每個資源的基本作業方式。

●設計者(Designer)

對於軟體安全議題設計者的主要職責為盡可能地將軟體的安全風險降低，面向包含：

– 找出滿足安全需求之技術，並深入研究該技術以決定如何正確使用之。

– 當軟體的安全弱點被發現，通常由設計者評估狀況並決定如何處理問題。

– 設計者需要協助支援軟體安全品質的評量。一般來說，包含提供用於評量或作為安全評鑑基礎的資料。

●實作者(Implementer)

在高度結構化的軟體發展環境，多數的實作者根據規格進行軟體建置工作並與設計者溝通未有明確文件紀錄的相關考量。在這樣的環境中，實作者的安全責任已經相當小，主要需依照程式碼撰寫規範及安全需求設定的水準，進行軟體的開發。這樣的方式使得第三方稽核能夠較容易的決定，該軟體是否已經如同預期般地安全。

●測試分析者(Test Analyst)

在結構化的開發團隊，軟體安全不應對測試流程有重大影響。測試人員仍然針對需求進行測試，實施迴歸測試等等。實務上，對於軟體安全性通常需要新的測試工具，因為針對功能性或效能的傳統測試工具並不能找出安

本文件之智慧財產權屬行政院資通安全辦公室所有。

全風險。測試團隊人員不必要是安全專家，他們需要的是自動化工具及風險的相關訓練，以找出安全問題。

●安全稽核者(Security Auditor)

安全稽核者的基本職責是檢查專案的目前狀態並試著確保專案的安全性。當驗證安全需求時，稽核者會決定實作結果是否滿足安全需求。當驗證安全設計時，稽核者主要決定設計是否影響安全性，產生軟體漏洞。當驗證安全實作時，稽核主要企圖發現明顯的安全問題，來自於實作與規格的偏差。

2.3.安全軟體發展相關標準介紹

本節介紹與安全軟體發展流程相關的三個標準。ISO/IEC 27001:2013 為目前國際上最廣泛採用之資訊安全管理規範，2013 年版新增與軟體安全相關的 A.14 控制措施項目，因此特別針對 A.14 部分進行摘要說明並將其控制措施項目對應本指引各章節，俾利對照參考使用。ISO/IEC 27034:2011「應用程式安全」(Application Security)則是微軟實行安全發展生命週期(Security Development Lifecycle, SDL)多年後，進一步推動的國際標準。NIST SP 800-64「資訊系統發展週期的安全考量」，為美國國家標準與技術研究院所推出的標準，針對系統開發生命週期所需要注意的安全事項進行探討。

2.3.1. ISO/IEC 27001:2013

資訊安全管理系統之定義為，全面管理系統的一部分，依據企業風險導向，建立、實作、運作、監視、審查、維持及改進資訊安全[5]。ISO/IEC 27001:2013 則是目前國際上最廣泛採用之資訊安全管理規範。國際標準組織(ISO)於 2013 年公布最新版 ISO/IEC 27001:2013，其中增加了 A.14 有關系統發展的

本文件之智慧財產權屬行政院資通安全辦公室所有。

相關控制措施，名稱為「系統獲取、開發及維護」。以下將就其內容進行概要說明並於表 2 列舉本指引相關章節以供參考。

●A.14.1 「資訊系統之安全要求事項」

確保資訊安全係跨越整個生命週期之整體資訊系統的一部分。此亦包括經由公共網路提供服務之資訊系統的要求事項。

– A.14.1.1 「資訊安全要求事項分析及規格」：資訊安全相關要求事項，應納入新資訊系統或既有資訊系統之強化的要求事項中。

– A.14.1.2 「保全公共網路之應用服務」：應防範於公共網路上傳送的應用服務中涉及之資訊，免於詐欺活動、契約爭議及未經授權揭露與修改。

– A.14.1.3 「保護應用服務交易」：應保護應用服務交易中涉及之資訊，以防止不完整傳輸、誤選路(Mis-routing)、未經授權之訊息修改、未經授權之揭露、未經授權之訊息複製或重演。

●A.14.2 「於開發及支援過程中之安全」

確保於資訊系統之開發生命週期內，設計及實作資訊安全。

– A.14.2.1「保全開發政策」：應建立軟體及系統開發之規則，並應用至組織內之開發。

– A.14.2.2「系統變更控制程序」：應藉由使用正式之變更控制程序，以控制開發生命週期內之系統變更。

– A.14.2.3「運作平台變更後，應用之技術審查」：當運作平台變更時，應審查及測試營運之關鍵應用，以確保對組織運作或安全無不利衝擊。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– A.14.2.4「軟體套件變更之限制」：應不鼓勵修改軟體套件，且僅限於必要變更，並應嚴格控制所有變更。

– A.14.2.5「保全系統工程原則」：保全系統之工程原則，應予建立、文件化、維持及應用於所有資訊系統實作工作。

– A.14.2.6「保全開發環境」：對涵蓋整個系統開發生命週期之系統開發及整合工作，組織應建立並適切保護安全開發環境。

– A.14.2.7「委外開發」：組織應監督及監視委外系統開發活動。

– A.14.2.8「系統安全測試」：於開發中，應實施安全功能性之測試。

– A.14.2.9「系統驗收測試」：應建立新資訊系統、系統升級及新版本之驗收測試計畫及準則。

●A.14.3 「測試資料」

確保測試用資料之保護。

– A.14.3.1「測試資料之保護」：應小心選擇、保護及控制測試資料。

表2　ISO/IEC 27001:2013 附錄 A.14 控制措施與本指引章節關係對應表

A.14 系統獲取、開發及維護		
A.14.1 資訊系統之安全要求事項		
A.14.1.1	資訊安全要求事項分析及規格	本指引相關章節： 3.1 安全軟體需求
A.14.1.2	保全公共網路之應用服務	本指引相關章節： 3.1 安全軟體需求 3.3.1 常見軟體漏洞及控制措施

本文件之智慧財產權屬行政院資通安全辦公室所有。

A.14.1.3	保護應用服務交易	本指引相關章節： 3.1 安全軟體需求 3.3.1 常見軟體漏洞及控制措施
A.14.2 於開發及支援過程中之安全		
A.14.2.1	保全開發政策	本指引相關章節： 2.4.1 策略與評量 2.4.2 政策與法規遵循
A.14.2.2	系統變更控制程序	本指引相關章節： 3.5.2 變更管理
A.14.2.3	運作平台變更後，應用之技術審查	本指引相關章節： 3.5.1 強化部署環境
A.14.2.4	軟體套件變更之限制	本指引相關章節： 3.5.2 變更管理
A.14.2.5	保全系統工程原則	本指引相關章節： 3.2.1 安全軟體設計法則 3.2.2 軟體設計風險分析
A.14.2.6	保全開發環境	本指引相關章節： 3.3.3 安全開發環境
A.14.2.7	委外開發	本指引相關章節： 3.6 安全軟體委外開發管理
A.14.2.8	系統安全測試	本指引相關章節： 3.4.1 源碼檢測 3.4.2 滲透測試
A.14.2.9	系統驗收測試	本指引相關章節：

本文件之智慧財產權屬行政院資通安全辦公室所有。

		3.4.3 安全需求驗證
A.14.3 測試資料		
A.14.3.1	測試資料之保護	本指引相關章節： 3.4.4 測試資料保護

資料來源：本指引整理

2.3.2. ISO/IEC 27034:2011

微軟推動安全開發生命週期(SDL)至今，導入標準化的作業為現階段的主要任務，故微軟致力於推動基於國際標準之 ISO/IEC 27034:2011，協助企業確保開發安全時候亦兼顧業務營運層面之考量。

ISO/IEC 27034:2011 為第一套針對軟體開發安全性所需之流程與框架之國際標準，除提供了通用的驗證型態，並未採用安全開發流程框架的作法，提供簡單扼要之大綱，也提供了採購者，得以要求各產業、各平台與不同區域之供應商遵循與落實軟體安全開發，亦協助開發商將安全軟體生命流程精神導入日常業務中。

ISO/IEC 27034:2011 包含 6 個部分，目前僅正式釋出 ISO/IEC 27034-1:2011，此部分定義 ISO/IEC 27034:2011 並非為一軟體開發標準、軟體專案管理標準，更非為軟體開發生命週期標準，主要希望針對軟體開發生命週期提供一實作之指引，包含[6]：

- 針對軟體安全制定流程作為應用程式系統於設計、開發、測試、實施和維護過程中對於功能安全的控制遵循使用，提供組織選用流程中合適的控制措施與衡量指標作為軟體開發的風險管理使用。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●提供與測試計畫概念相符之信任層級(Target Level of Trust)，作為應用程式之設計、建置與驗證使用。

●提供政府與軍事系統如 Common Criteria 方案之稽核與驗收通用標準。

其中組織規範架構(Organization Normative Framework, ONF) 用來協助組織將現有架構進行調整，ONF 包括了相關的政策、程序、角色和工具套件與最佳實踐項目供組織使用，ONF 項目如下：

●Lifecycle Reference Model。

●Application Security Control(ASC) Library：包含組織已核准之相關安全控制措施，此控制措施來自於應用程式安全風險識別、資產影響與對應之安全信任等級而產生。

●Roles, Responsibility and Qualification: 包含所有的人員角色對應與能力適用性。

●Application Specification。

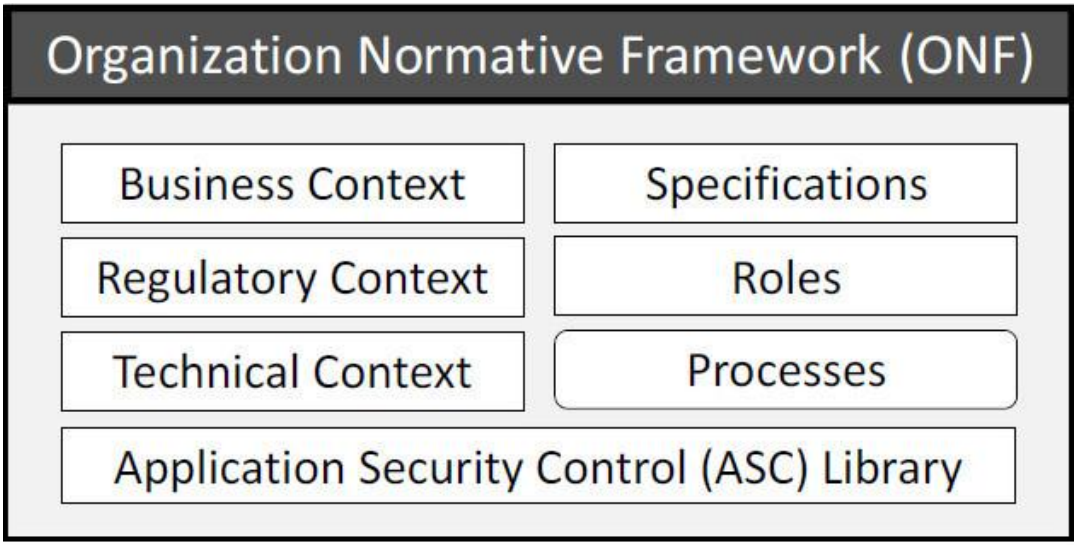
●Technological Context: 包含應用程式安全所有安全需要與其對應的相關技術。

●Regulatory Context：包括所有與應用程式安全相關的的標準，法律與法規。

●Business Context：包含組織所有應用程式開發的安全策略，標準和最佳實踐

有關組織規範架構(Organization Normative Framework, ONF)，詳見圖 2 所示。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[6]

圖2　Organization Normative Framework

而應用程式標準規範(Application Normative Framework, ANF)可視為 ONF 的衍生，應用程式應套用組織規範框架並分析後，選擇適合該應用程式所欲遵循的安全標準層級，由於組織擁有不同類型之應用程式，故組織應透過應用程式安全管理流程(Application Security Management Process, ASMP)產生與維護各種類型與安全信任層級之應用程式規範架構供使用，應用程式安全管理流程包含下列 5 步驟，詳見圖 3 所示：

●指定應用程式的要求和環境。

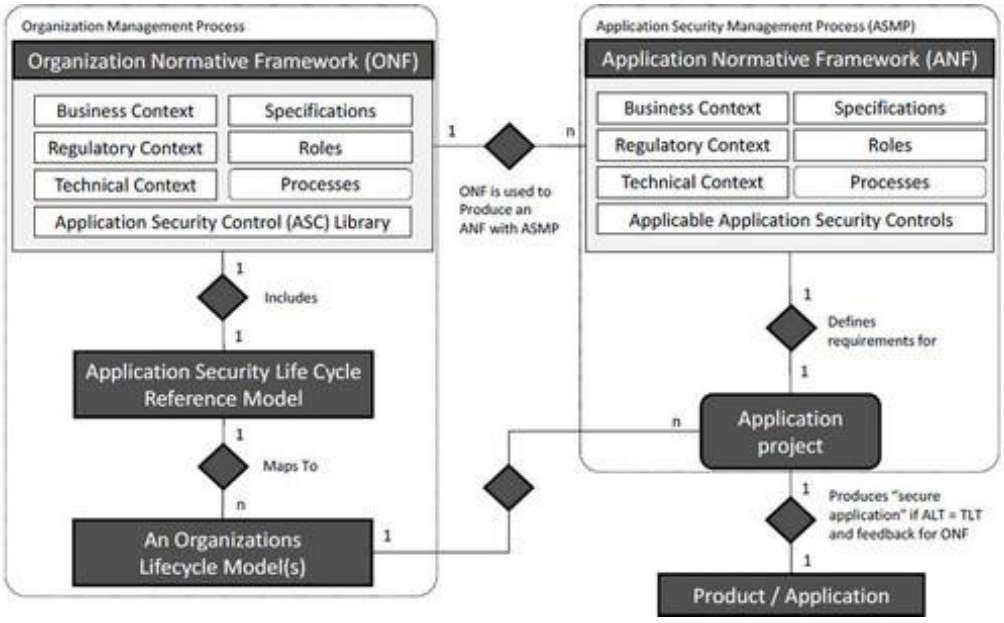
●評估應用程式安全風險。

●建立和維護應用程式的規範性框架。

●應用程式的部署與操作。

●應用程式安全性的監控。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[6]

圖3　Application Security Management Process 關係圖

整體而言 ISO/IEC 27034-1:2011 制定管理組織規範架構的流程，透過建立雙向的溝通管道達到組織之持續營運。

ISO/IEC 27034:2011 雖不與 ISO/IEC 27001:2013 規範相依，但其補足 ISO/IEC 27001:2013 中未具體提及的應用程式安全性實務守則，故 ISO/IEC 27034:2011 如 ISO/IEC 27002:2013 之定位相似，可成為實踐 PDCA 品質保證之參考方案。

而 ISO/IEC 27034:2011 附錄中提供了與組織安全架構相容之微軟的 SDL 供使用，另外如 OWASP 組織也提供與 ISO/IEC 27034:2011 對應之相關軟體安全項目，如 ISO/IEC 27034:2011 與 Open SAMM 框架之對應關係，詳見圖 4 所示。

本文件之智慧財產權屬行政院資通安全辦公室所有。

Open Software Assurance Maturity Model		ISO/IEC 27034									
		Normative Framework							Application Security Risk Assessment	Provisioning and Operating theApplication	Application Security Audit
Function	Security Practice	Business context	Regulatory context	Technological context	Application specifications repository	Roles, responsibilities andqualifications	Application security control library	Life cycle reference model			
Governance	Strategy & Metrics	◆						◆	◆		◆
	Policy & Compliance	◆	◆				◆				◆
	Education & Guidance			◆		◆	◆				
Construction	Threat Assessment				◆		◆		◆		
	Security Requirements				◆		◆			◆	
	Secure Architecture			◆	◆		◆			◆	
Verification	Design Review						◆			◆	
	Code Review						◆			◆	
	Security Testing						◆			◆	
Deployment	Vulnerability Management			◆			◆			◆	
	Environment Hardening						◆			◆	
	Operational Enablement				◆		◆			◆	

資料來源：[6]

圖4　ISO/IEC 27034:2011 與 Open SAMM 對應關係

2.3.3. NIST SP 800-64

美國國家標準與技術研究院(National Institute of Standards and Technology, NIST)提出特刊(Special Publication, SP) 800 系列是有關於資訊技術的安全標準。其中 SP 800-64 資訊系統發展週期的安全考量(Security Considerations

本文件之智慧財產權屬行政院資通安全辦公室所有。

in the Information System Development Life Cycle)，是針對系統開發生命週期所需要注意的安全事項進行探討。其協助機構建置基本的安全活動，將其整合至系統發展生命週期(System Development Life Cycle, SDLC)的建立，並能更符合成本效益、風險適當的安全控制識別、開發與測試[7]。

2.3.3.1. 簡介

系統發展生命週期提供許多的方法協助組織有效地開發資訊系統。根據系統預期的大小與複雜度、開發時程，以及系統使用時間的長度，選擇適合的系統發展生命週期模型來使用與系統開發。SP 800-64 指引採用系統發展生命週期的線性順序模型(Linear Sequential Model)來將安全納入，因其對各種模型來說，是一個最簡單且合適的討論平台，而所討論的概念也可適用於任何的系統發展生命週期模型。

此外，與其他資訊技術建立具共識的系統發展生命週期，並討論系統發展生命週期內的角色與參與資訊安全整合的責任。提供資訊系統開發時，關鍵安全角色與其職責的描述，協助不熟悉系統開發生命週期流程的人員，可以有足夠的資訊了解有關資訊安全與系統發展生命週期之間的關係。

設計一個考量安全的解決方案，可大幅減少額外安全控制的需求，也使得企業的安全規劃可以重複使用，並降低成本和開發時程，促進安全的可信賴性。實現系統開發生命週期內的安全整合最有效的方法是規劃與實行全面的風險管理計畫。這將使得整合安全的成本與需求，以及一個嵌入式且重複授權的流程，能在整個機構內提供風險資訊給資訊技術的利益相關者與開發人員。

2.3.3.2. 將安全整合至系統發展生命週期

本文件之智慧財產權屬行政院資通安全辦公室所有。

一個典型的系統發展生命週期包含五個階段，分別為初始(Initiation)、需求與獲取(Development/Acquisition)、建置與評鑑(Implementation/Assessment)、運作與維護(Operations/Maintenance)，以及汰除(Disposal)。在系統開發過程中，每個階段都需要有效的將安全納入最低限度的安全任務內，且有可能會在整個系統生命週期內持續的重複。有關系統發展生命週期五個階段的示意圖，詳見圖 5 所示。



資料來源：[7]

圖5　NIST SP 800-64 系統發展生命週期示意圖

SP 800-64 指引提供系統發展生命週期各個階段實用的文件，包含主要的活動與里程碑；決策點或控制閘；在系統設計時提供重要資訊的預期輸出；專案成果；以及系統維護、安全和運作方面的考量等。專案初期實行資訊安全允許需求因應所需而成熟與整合，以符合成本效益。

2.3.3.2.1. 系統發展生命週期：初始階段

本文件之智慧財產權屬行政院資通安全辦公室所有。

初始階段是系統發展生命週期的第 1 階段，其主要安全活動包括：在機密性、完整性與可用性方面初期劃分其業務需求；針對傳輸、儲存或建立的資訊，確定其資訊分類並識別已知且需特別處理的需求，如個人識別資訊；確定任何的隱私要求。

透過適當的風險管理規劃，盡早地規劃與評估其成本和時間。讓開發人員可在專案內計畫其安全需求和相關限制；也提醒專案負責人，隨著專案的持續進行，仍會有許多的決策是需要適當的權衡其安全影響。安全的討論應作為開發專案不可分的一部分來執行，以確保企業策略的專案人員與其整體開發專案的風險影響有完整的理解。

一般包含的控制閘有：決定整個開發過程中，其餘部分使用的獲取策略；系統概念的審查，驗證該概念是可行的、完整的、可實現的，並本著組織的任務目標與預算限制；性能規格審查，確保系統設計初期解決當前確定的所有特定安全要求；企業架構(Enterprise Architecture, EA)的取向，協調資訊技術的願景、標準，與業務需求，以及當前和即將發生的安全服務其安全定位；財務審查，驗證該系統在平衡風險管理相關的成本問題時，資本規劃和投資控制(Capital Planning and Investment Control, CPIC)構件和指引的一致性；風險管理審查，符合 NIST 風險管理架構指引的建議，減少管理系統風險的模糊性。這個風險管理審查包含審查資訊系統安全分類結果，其中包括識別資訊類型、結果影響層級，和最終系統的安全性分類。

2.3.3.2.2. 系統發展生命週期：需求與獲取階段

需求與獲取是系統發展生命週期的第 2 階段。主要的安全活動包括：進行風險評鑑，並使用其結果來補充安全控制的基準線；分析安全需求；執行功能與安全測試；準備系統認證與認可的初期文件；設計安全架構。

本文件之智慧財產權屬行政院資通安全辦公室所有。

雖然此階段呈現的資訊安全元件其順序是自上而下的(Top-down)方式,但完成的順序不一定是固定的。複雜系統的安全分析有可能重複，直到達成其一致性與完整性。另外，系統開發流程的文件，應配合安全需求分析、安全架構、業務影響評估，以及安全分類，透過綜合的方法來提升其決策制定的能力，達到成本節省與差距檢測。

一般包含的控制閘有：架構或設計審查，用以評估規劃的系統設計以及潛在整合共享服務(Shared Service)與共用安全控制的其他系統,如身分認證、災害復原、入侵偵測，或事故報告；系統效能審查，評估系統是否能交付或有能力交付擁有者期望的文件，以及當其受到不當使用時，系統的行為是可預見的方式。例如，系統在預期資源負載極端的情況下，維持其可用性與資料完整性的能力；系統功能審查，確保功能需求的識別夠詳細且具可測試性；專案中的狀態以及財務的審查，在工作量估計規劃層級中偵測其重大轉移是相當的重要,其確保成本效益比率的監控與有效決策的保持；如果有上述的審查，則當系統與(或)其安全控制，以及(或)需有變更時，將可能需要對風險管理決策進行後續的審查。

2.3.3.2.3. 系統發展生命週期：建置與評鑑階段

建置與評鑑是系統發展生命週期的第 3 階段，在此階段中，系統將被安裝並在該組織的營運環境內進行評鑑。主要的安全活動包括：整合資訊系統納入其環境；與安全控制測試同步的計畫與實行系統認證活動；完整的系統認可活動。

預期產出的初期工作計畫，可用於識別關鍵人員、專案限制、核心元件、測試範圍，和層級預期嚴謹的計畫文件。且相關的認證套件(Certification Package)應是接近於完整，並在一致性的審查下開始實施。

本文件之智慧財產權屬行政院資通安全辦公室所有。

一般包含的控制閘有：系統測試準備審查；認證與認可(Certification and Accreditation, C&A)的審查；最後的專案狀態及財務審查；部署準備審查；授權官(Authorizing Official, AO)的決定；資訊技術部署或連接核准。

2.3.3.2.4. 系統發展生命週期：操作與維護階段

操作與維護是系統發展生命週期的第 4 階段。此階段的系統應到位(In-place)並運行、增強和(或)修改系統的開發與測試，以及硬體和(或)軟體的添加或更換。該系統依據其安全需求，進行系統所需的修改整合，並監控其持續性的效能。操作系統被週期性地評鑑以測定系統如何可以更有效益、安全，和有效的。主要的安全活動包括：進行操作準備的審查；管理系統的組態；制定流程、確認操作程序，以及持續監測資訊系統的安全控制，如安全審查、自我評估、組態管理、防毒管理、修補管理等；根據不同情況進行重新授權。

當系統轉移到生產環境中時，如果系統發生無計畫性的變更是顯著的話，則安全控制的修改測試，如組態，就可能需要確保其安全控制的完整性。此步驟並不一定需要。但有助於降低風險，並有效地解決突發的意外。

此階段一般包含的控制閘有：操作準備的審查；變更控制委員會審查建議的修改；審查行動與里程碑計畫(Plan of Action and Milestones, POA＆M)；認可決策(每 3 年或主要系統變更後)。

2.3.3.2.5. 系統發展生命週期：汰除階段

系統發展生命週期的最後一個階段是汰除，其提供系統的汰除與任何契約的結案。確保系統的有序終止，並保存有關系統的重要資訊，使一些或所有的資訊可以在未來有必要的情況下重新啟用。尤其是資訊安全議題相關的資訊與系統汰除應該被明確的處理。

本文件之智慧財產權屬行政院資通安全辦公室所有。

主要的安全活動包括：建置與執行汰除或轉移計畫；關鍵資訊的檔案保管處；媒體清除；汰除硬體和軟體。建立汰除或轉移計畫，確保所有利益相關者都知道系統及其資訊的所有關鍵元件、服務，和資訊的汰除或轉移狀態，並確定必要的步驟、決策，和適時地關閉、轉移，或遷移系統或資訊的里程碑。

此階段一般包含的控制閘有：系統終止審查；變更控制委員會；終止安全審查。其中，適當的安全性文件檔案、更新聯邦資訊安全管理法(Federal Information Security Management Act, FISMA)的報告和企業架構的詳細目錄等，可協助系統順利的終止。

2.4.導入安全軟體發展流程之管理重點

組織導入安全軟體發展流程管理，主要聚焦於「策略及評量」、「政策及法規遵循」、「教育訓練」面向。「策略及評量」強調計畫、分派角色及責任、定義軟體安全目標、決定組織資源投入及定義評量方法及設定軟體安全檢查關卡。「政策及法規遵循」強調為了合乎相關法規需求，組織定義何種相關控制措施，來控制相關風險。「教育訓練」扮演組織中提升軟體安全重要性的角色，因為軟體開發人員通常需要提升資訊安全認知及相關知識。另外，組織為推動安全軟體發展流程，可考慮建立「軟體安全小組」，負責相關事務。各面向之相關安全活動詳述如下[8][9]。

2.4.1. 策略與評量

●發布流程文件(角色、職責、計畫)

處理軟體安全的相關流程與程序文件，公開發布給所有相關人，以使所有相關人員明瞭計畫。軟體安全目標、角色及其職責、相關活動被清楚的定

本文件之智慧財產權屬行政院資通安全辦公室所有。

義。大部分的組織會從公開的軟體安全發展流程方法論中採取所需要的部分實行，以符合組織需求。

●內部宣導

為了建立組織對於軟體安全的支援，軟體安全小組的成員之一通常扮演組織內部之軟體安全「傳教士」角色，負責內部宣導。「傳教士」角色可對內部群組進行演講、作為外部講者的邀請人、撰寫內部使用的白皮書，或建立一系列內部使用的軟體安全資源供內部宣導使用。

●管理階層支持

管理階層應了解有關軟體安全性不足的後果及對商譽、商業損失的影響。管理階層也應了解其他組織為了達成軟體安全所進行的作法。同時了解軟體安全問題及其正確的解決方案後，管理階層才會接受軟體安全作為組織風險進行管理的必要性。軟體安全小組可以展示一個可能發生最嚴重的情境，實際的顯示使用的攻擊方式及造成的商業影響。此一作法對於取得後續相關資源有所幫助。

●建立檢查點

軟體安全的檢查點會設置在多個軟體發展生命週期中的階段。

為建立軟體安全檢查點的初始步驟為：

– 定義檢查點在軟體發展流程中的位置。

– 該檢查點收集何種資訊以決定軟體開發是否可繼續向下進行。

例如，軟體安全小組可以在軟體發布前收集安全測試的結果，與組織定義的標準進行比較，以決定是否放行。先行定義檢查點但是不一開始就強制

本文件之智慧財產權屬行政院資通安全辦公室所有。

執行的作法，對於軟體開發團隊漸漸走向軟體安全的方向有所幫助，以避免軟體專案適應不良導致其他問題(例如時程延誤)。

●風險接受程序

組織需具備安全風險接受程序並記錄其歸屬責任。當軟體安全的剩餘風險大於原先設定的標準，卻採行接受其風險時，組織需要有特定審核程序並留下文件紀錄。

●組織內部發布軟體安全資訊

軟體安全小組於組織內部公開關於各軟體專案安全狀態的相關資料。有時候這樣的資料並非提供給組織內所有的軟體專案成員，而是相關的管理階層或經理人就足夠了。

●施行評量檢查點並追蹤例外

軟體安全的檢查點開始施行，軟體安全小組負責追蹤例外狀況。一個強制的檢查點可以要求專案團隊在通過前消除找到的嚴重問題。有些情況下，評量檢查點與規範或合約要求的控制措施相關，其他的情況下，檢查點通常作為表現的關鍵指標，用來管理流程。

●建立軟體安全事務的窗口

識別組織中個別單位的軟體安全事務窗口，是建立人際網絡以加速軟體開發安全的重要步驟。組織中個別單位的軟體安全事務窗口，對於軟體安全的技能與興趣，須在一般的水準以上。因此，一種方式為根據基礎軟體安全教育訓練中,表現較為積極或傑出的成員來挑選。若考慮涵蓋率的問題，則採取組織由上而下的方式，以指派的方式，挑選各單位的軟體安全事務

本文件之智慧財產權屬行政院資通安全辦公室所有。

窗口。在組織中各單位具有溝通良好的軟體安全事務窗口，對於初步開始發展軟體安全的組織來說，是一個良好徵兆。

●定義評量準則並驅動資源

軟體安全小組以及管理階層挑選軟體安全品質之評量指標。評量指標將會驅動組織初步投入預算和資源。評量指標也使得軟體安全小組得以採用量化方式明確解釋其目標。一個可能的評量指標是安全缺陷的密度(例如，每千行源碼中發現的安全缺陷數量)。安全缺陷數量的減少可用來顯示，隨著時間的推移，整治措施發揮的成效。此一步驟的關鍵在於將技術上的成果指標與商業目的結合，以使投入相關資源的成果被彰顯。因為管理階層透過相關教育訓練已具備軟體安全風險意識，作到此一結合對於成果顯現將會非常有幫助。

●根據風險對資料與軟體進行分級

建立簡易的分類系統，設立明確分類軟體風險等級的標準。此一分類可以是簡單的區分為高/中/低風險。對於資料而言，相似的分類系統也應該被發展，資料的風險等級是根據其對於組織的可能影響進行分級。根據軟體所收集資訊的風險等級，以及軟體本身的風險等級分類標準，來賦予每個軟體專案對應的風險等級。以上流程可以由軟體安全小組統一處理，或是由個別的軟體專案團隊自行透過填寫風險分級問卷來達成。將軟體專案/資料進行風險等級分類的動作應成為組織內持續進行的流程之一，而非一次性的動作。組織須確保新的軟體專案或收集的資料也進行了風險分級。一個可行的作法是，於軟體專案建案時進行此活動，並在發展流程各階段進行重新檢視。

2.4.2. 政策與法規遵循

本文件之智慧財產權屬行政院資通安全辦公室所有。

●法規遵循一致化

如果商業需求或是客戶面臨到法規遵循的限制，軟體安全小組就扮演了解法規限制並驅動軟體安全活動施行的關鍵角色之一。軟體安全小組應創造一致性的作法來遵循法規需求，並推動至全組織，以免除不同軟體專案需要各自了解法規遵循的重複工作。正式的作法是將法規適用的部分，對應到控制措施的陳述，以清楚說明組織如何遵守法規。作為替代方案，現行組織的流程若以其他單位，例如法務部門，作為法規遵循的關鍵主要負責單位亦為可行作法。此一安全活動的目標是創造一個指引，使得法規遵循的工作是有效率的(透過移除重複工作)。

●定義處理個資規範

軟體處理個資的方式應該被明確的規範。軟體安全小組將在組織內扮演識別軟體中使用個資的領導角色，以符合法規及客戶的需求。例如，如果軟體具有處理信用卡資訊的交易行為，軟體安全小組將識別法規的限制並對持卡人資料的處理提出限制與保護措施。需注意的是,採用使用雲端服務、主機代管等外包的方式並不能免除組織對於個資保護的責任。開發軟體產品的公司，該軟體若是預計會處理個資，也應提供隱私保護措施與相關指引給使用軟體的客戶。

●建立軟體安全政策

軟體安全小組透過建立或協助建立軟體安全政策來指引組織滿足法規或來自客戶的安全需求。軟體安全政策文件會著重在法規遵循的議題，例如個資的處理以及為保護敏感資料而進行的密碼學應用。政策文件可直接說明組織將如何採用安全軟體發展流程。

●個資識別

本文件之智慧財產權屬行政院資通安全辦公室所有。

組織識別所有系統內部儲存的個資種類與所在位置。要識別所有個資的所在位置，有兩個方法：每個軟體自行標示出所有使用到的個資種類，或透過檢視個別種類的個資，其整個利用的流程有哪些軟體會使用他。

●對法規遵循實作控制措施並追蹤

進行「法規遵循一致化」程序後，由於法規遵循對應至特定一或多個控制措施，組織可以展現實務作為已符合控制措施，因此而達成法規遵循。軟體安全小組追蹤控制措施的符合程度，以確保稽核者對於控制措施感到滿意。如果組織的安全軟體發展流程尚未發展完善，軟體安全小組在此項目要扮演較重要角色。

●供應商合約規範

軟體發展流程中相關元件或外包的供應商，其合約應該包含有關軟體安全的服務層級協議(Service Level Agreement, SLA)，以確保供應商不會危害到整體的軟體安全程度及組織符合的法規規範。每一份新的或更新的合約應該包含要求供應商產出的產品或服務，必須符合組織的軟體安全政策。

2.4.3. 教育訓練

●安全意識教育訓練

軟體安全小組提供安全意識的訓練，以在組織中推廣注重軟體安全性的文化。教育訓練可以由軟體安全小組成員提供、由組織外部教育訓練機構或是透過數位學習課程提供。安全意識訓練課程不需要對特定人士量身訂製，例如，所有的程式設計師、品保工程師、專案經理可以參加相同的軟體安全初階課程。安全意識課程如果僅針對開發者而不提供給其他組織內相關角色是不足夠的，應該涉及組織內所有與軟體專案相關的角色。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●以專案角色為基礎之教育訓練

相較安全意識教育訓練，軟體安全訓練更進一步地幫助軟體專案相關角色將安全實務整合進日常工作中。教育訓練是根據角色客製化的，受訓者可以學習到相關工具的資訊、技術以及與本身最相關的安全弱點類型。組織對於工程師可以提供針對軟體架構、程式語言類型及測試者的不同類型之訓練課程。特定工具的訓練也相當常見於軟體安全訓練課程中。需考量一項課程可能對於組織中多種不同角色有所幫助，例如程式設計師也需要了解如何進行軟體測試。

●歷史事件

為了對組織創造強烈且持久的行為變化，訓練課程應該包含組織過往的軟體安全相關歷史事件。當受訓者可以看到自身的角色於過往問題中所身處的位置，才能夠更好的了解訓練教材與自身工作的相關性，以及如何將所學應用到工作上。一個方法是在課程中使用組織過往被攻擊的案例當作實際的教材範例。組織過往的歷史案例必須仍然與現今的作業有相關性，才對課程內容有正向引導的作用。

●透過教育訓練強化聯繫

軟體安全小組透過舉辦特殊活動來強化軟體安全事務窗口間的聯繫。可採用一同學習進階議題或邀請外部講者的方式。電話會議不能夠替代此類活動，即便是一年數次面對面的活動也可加強各單位窗口間的聯繫。

●報到程序中提供安全資源

組織中有關軟體專案的新進人員，其報到程序中，包含軟體安全相關的資源或教材。一般的新進人員，其報到程序中，也應包含基本的安全意識訓

本文件之智慧財產權屬行政院資通安全辦公室所有。

練教材，例如密碼複雜度原則、如何取得組織內部安全相關資源等。此作法的目的是確保新進人員也了解組織內注重安全的文化。

本文件之智慧財產權屬行政院資通安全辦公室所有。

3. 安全軟體發展流程階段實務

3.1.安全軟體需求

軟體安全性一般都被列為事後考慮的情況，通常當軟體發生安全事故，或者當功能需求在專案資源範圍內被滿足時，軟體安全性才會被列入考量，使得軟體安全品質的問題愈加嚴重。

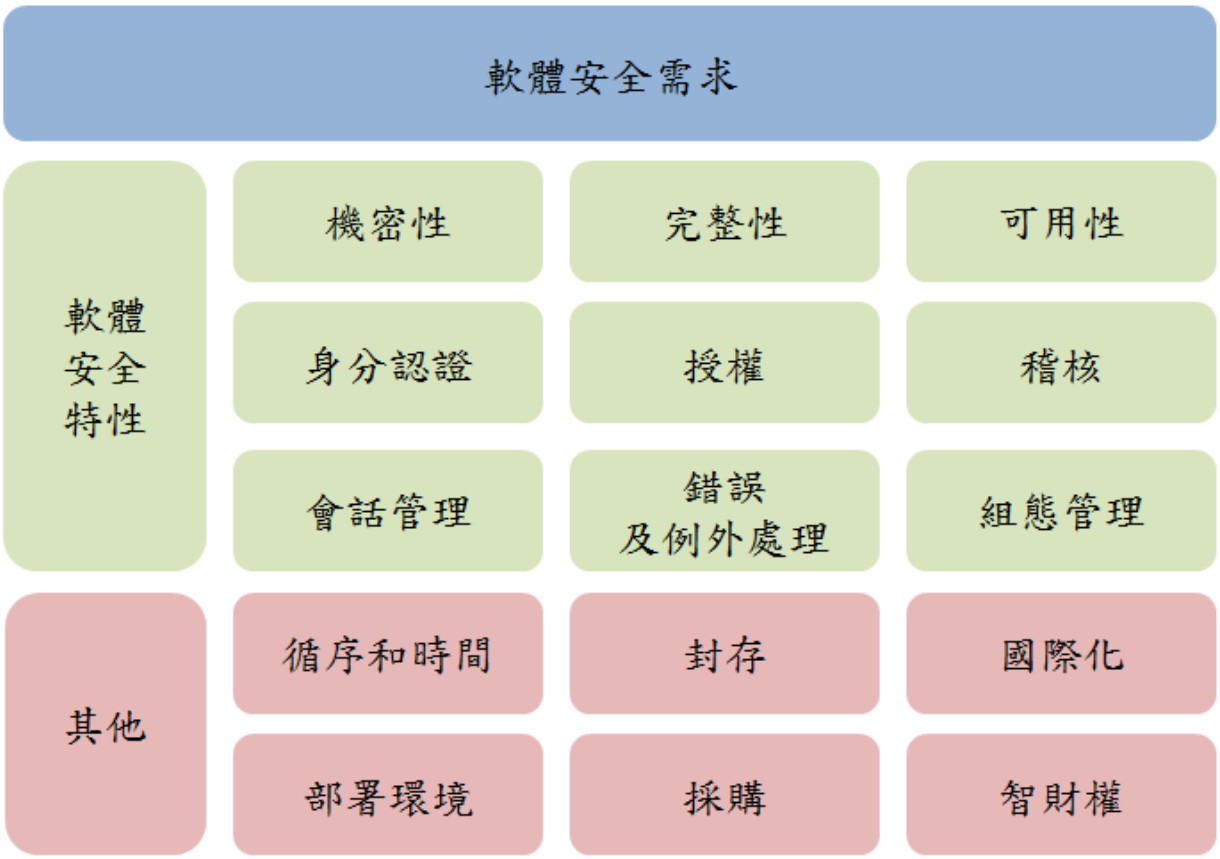
安全軟體發展流程則具備事前安全性的思維，採用安全軟體發展流程建置軟體安全性，代表須從流程初期需求階段就考慮安全性相關的需求。安全需求最主要的差異在於，針對保護資訊資產、功能以及避免軟體被誤用的方向進行考量。

以下各小節將依序說明軟體安全需求類型的種類,如何進行安全需求萃取，以及用於後續追蹤需求的方法：「需求追溯矩陣」。

3.1.1. 軟體安全需求類型

可大致區分為來自「軟體安全特性」及「其他」兩大類[10]，詳見圖 6 所示，分別說明如下。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[10]

圖6　軟體安全需求類型

●機密性

防範資料的洩露。機密性需求是私有或機密的資訊不被洩露給非授權人員[11]。機密性需求可分為資料 1.儲存時 2.傳送時 3.封存時。

●完整性

保護資訊或程序不被故意或意外的非授權變更。

●可用性

本文件之智慧財產權屬行政院資通安全辦公室所有。

可用性是確保授權使用者需要時可以存取系統或服務。

●身分認證

確認實體(Entity)要求身分之真實性的程序。實體可能是人、程序或硬體裝置等。身分認證要求實體提供憑據以證明其確實為其聲稱的主體。

●授權

確認已認證的實體是否被允許對特定資源進行特定動作的程序。

●稽核

協助建構出使用者行為的歷史紀錄。軟體須可以提供誰、做了什麼、在什麼地方與何時等資訊。

●會話管理

會話(Session)是用來維持軟體中的狀態，但是也對軟體安全性造成影響。成功的身分認證後，會話的識別碼(Session Identifier，Session ID)被發給使用者，Session ID 被用來追蹤使用者，維持其已進行身分認證的狀態，直到其身分認證失效或會話過期。無狀態的協定(例如，HTTP)沒有會話則使用者每一次進行動作，就必須重新進行身分認證，這樣的作法通常無法被使用者接受。然而，使用會話的作法，又面臨 Session ID 如果被竊取，則有被身分偽冒的風險，因此必須規劃安全的會話管理方式。

●錯誤及例外處理

錯誤(Error)及例外(Exception)是資訊洩漏的潛在來源之一。詳細的錯誤訊息和未處理的例外報告，可能會導致洩漏內部應用程序架構、設計和設定

本文件之智慧財產權屬行政院資通安全辦公室所有。

的資訊，因此必須在安全需求中明確說明處理錯誤及例外的要求，以避免資訊洩漏威脅。

●組態管理

組態管理是用來建立或維持軟體功能、效能及相關特性的一致性和穩定。軟體所依賴的底層元件，須注意其安全組態設定以確保較高的安全性。對於軟體的設定參數，應加以保護並管理其變更。

●循序(Sequencing)與時間

軟體中關於循序與時間的設計失誤，將導致競賽情況(Race Condition)與檢查時間/使用時間(Time of Check/Time of Use)這類問題，因此相關需求應被注意並加入需求規格中。

●歸檔封存(Archiving)

組織若因為營運持續、內部政策或法規要求，需要將資料作歸檔封存動作，須注意保護資料的相關安全需求。

●國際化

國際化是設計一個軟體或系統，使其未來不需要重大軟體工程變更，便可以適用於各種語言和地區的過程。軟體因應國際化需求，將會有語系及時區的問題需要克服，在這個修改過程當中可能會產生安全問題，需要特別注意。

●部署環境

本文件之智慧財產權屬行政院資通安全辦公室所有。

現今軟體大多依賴其他底層元件或中間軟體才得以執行，因此其部署環境的安全將影響整體安全性。須注意部署環境的安全設定、定期更新及減少受攻擊面等安全議題，並產出相對應的安全需求。

●採購

組織評估效益及專業能力等問題，可能採取以採購的方式取得所需的軟體。除了功能需求外，應考量安全需求並將其包含在法律保護機制，如合約和服務層級協議(Service Level Agreement, SLA)中。

●智財權

若組織開發的軟體將作為商用軟體銷售，則需要特別注意智慧財產權保護方面的安全需求，例如程式碼混淆(Code Obfuscation)、程式碼簽章(Code Signing)、防止竄改(Anti-tampering)等機制。

3.1.2. 需求萃取方式(Protection Needs Elicitation, PNE)

3.1.2.1. 來自軟體安全政策的需求

軟體安全需求的其中一個來源是內部的政策。因為政策通常包含高階的、商業上的需求，因此需要被分解或細化為詳細的安全需求。然而此程序的高階需求來源並不只限於內部政策，外部法規遵循的需求也同樣在範圍中。若組織已有較明確的軟體安全政策，為遵循法規或客戶需求而將高階需求對應至特定一或多個控制措施，則發展詳細的軟體安全需求將會簡單許多。

3.1.2.2. 資料分級

在安全領域中，機敏資料或資訊通常被認為是重要的資訊資產，因而需要特別的保護措施。但是並非所有的資料都需要相同程度的保護，可以公開

本文件之智慧財產權屬行政院資通安全辦公室所有。

的資料甚至可能不需要任何保護措施。因此，資料分級就是考量資料被洩漏、竄改、破壞後的影響而賦予資料敏感程度的行為。組織應建立資料分級的準則，使軟體若蒐集、處理、利用機敏資料時，能據以判別資料的風險等級。資料分級後的結果可以被用來決定相對應的軟體安全需求。資料分級的主要目的是降低資料保護的成本，最大化投資報酬率。意即，根據資料的等級實施相對應程度的安全控制措施。組織可以規範資料分級為特定等級時，於不同的安全特性類別，例如機密性、完整性、可用性等，規範適用的共通或特定控制措施。

3.1.2.3. 軟體安全問卷

問卷對於收集軟體安全需求是相當有效率的方法。問卷應該同時包含明確特定的問題與開放式的問題。開放式的問題可獲得更多此軟體安全相關的資訊，但其結果不容易有一致性的評判。問卷問題應該依循軟體安全特性與安全設計法則的分類發展，其結果可以直接產生出軟體安全需求。

軟體安全問卷蒐集軟體的基本資訊以及各面向的狀況，可用來進行風險等級分類。對分類結果為高風險的軟體，後續應有更深度的架構檢視分析。軟體安全問卷之範例請詳見附件 1。

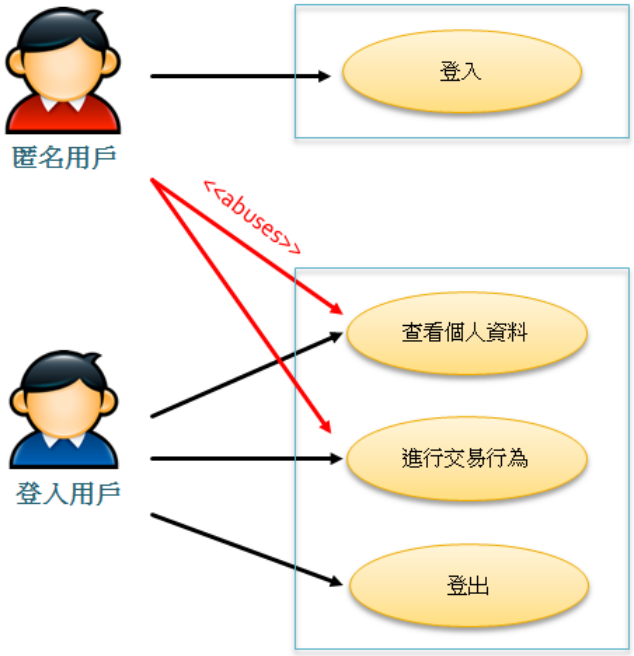
3.1.2.4. 誤用案例模型(Misuse Case Modeling)

從使用案例可發展出誤用案例，透過發展負面的使用情境來幫助識別安全需求，負面的使用情境是系統中意料之外的行為，為系統擁有者不希望發生的情況。藉由採用攻擊者的角度，誤用案例反轉了原先使用案例中正常使用者的視角，提供發覺外部威脅如何發生在軟體或系統中之機會。誤用案例的建模方式相似於使用案例，兩者的繪製方式相同，但是誤用案例記

本文件之智慧財產權屬行政院資通安全辦公室所有。

錄的是刻意或意外的狀況下，誤用者如何以非原本預期的方式與軟體進行互動。

誤用案例的例子如圖 7 所示，透過加入匿名使用者(攻擊者)之角色存取一般使用者登入後之正常使用行為，制定出該使用案例的誤用案例。



資料來源：[12]

圖7　誤用案例範例

3.1.2.5. 主體-物件矩陣(Subject – Object Matrix)

當軟體中具有多個主體(或角色)需要存取功能或物件，釐清每個主體(角色)被允許存取的功能與物件是非常重要的。物件是主體在軟體中可以互動的項目。高階的物件若可以被細化成細部的物件，將可以更明確的表示出主體對物件關係的正確性。例如，資料庫物件可以被細分為資料表格、資料表格視角(View)、預儲程序等，每種物件又會有多個實體。主體-物件矩陣

本文件之智慧財產權屬行政院資通安全辦公室所有。

中也應該正確地表示第三方元件與主體間的關係。主體-物件矩陣用來識別出允許的行為，可以幫助誤用案例模型的發展。

3.1.2.6. 腦力激盪

腦力激盪是最快也最非結構化產生軟體安全需求的方式。於此過程中，成員不需要闡述提出該軟體安全需求背後的原因，而是持續的記錄所有成員想到的軟體安全需求。雖然此快速產出軟體安全需求的方式相當符合現今軟體需要快速發展的環境，卻不建議僅採取此種作法。

腦力激盪有以下幾個缺點：

- 有很高的可能性，提出的軟體安全需求並不直接與組織的商業決策需求或是軟體的技術背景、安全背景吻合。
- 很容易有所遺漏，忽略某些重要的安全考量。
- 因個人想法是相當主觀的，通常結果不全面或是不一致。

腦力激盪可以作為先期軟體安全需求產出的方式，但是需要進一步採用其他更加結構化的方式來確保最終結果的全面性。

3.1.2.7. 來自設計階段活動的回饋

安全軟體設計階段的活動，其結果也應回饋並修正安全需求：

- 威脅建模(請詳見 3.2.2.1 節)

  威脅建模採用系統化的方法，以攻擊者的角度，識別可能影響軟體系統的威脅並進行評估。基於對架構與實作的了解，識別與評估威脅後，以風險高低的順序對威脅發展適當的控制措施。

- 架構風險分析(請詳見 3.2.2.2 節)

本文件之智慧財產權屬行政院資通安全辦公室所有。

架構風險分析可以分成三個主要的子流程：

－抗攻擊能力分析(Attack Resistance Analysis)。

－模糊分析(Ambiguity Analysis)。

－底層框架弱點分析(Underlying Framework Weakness Analysis)。

架構風險分析找出的安全問題與缺陷，應發展對應的控制措施。

●攻擊面分析(請詳見 3.2.3 節)

攻擊面分析的結果，若有可以減少權限或存取性的地方，應調整相對應的設計與需求。

●安全設計審查(請詳見 3.2.4 節)

分析設計中不安全的部分與缺乏的安全設計後，相對應的安全需求列表應該被建立並被回饋到「需求追溯矩陣」中。

3.1.3. 需求追溯矩陣(Requirements Traceability Matrix, RTM)

各種需求萃取方式(PNE)產出的軟體安全需求，可以放入需求追溯矩陣(Requirements Traceability Matrix, RTM)進行記錄與後續追蹤。需求追溯矩陣是一個記錄並對應商業需求、功能需求、測試案例編號等需求相關資訊的表格。它可以被用來記錄軟體安全需求並追蹤其後續處理狀態。

對軟體開發流程來說，需求追溯矩陣提供以下好處：

●確保沒有範圍認知上的落差。

●確保後續的設計會滿足特定的安全需求。

●確保實作不會偏離安全設計。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●提供定義測試案例的基礎。

需求追溯矩陣是一種對應需求及後續活動，以方便軟體專案追蹤後續需求是否真正被落實的概念，並非固定採取紙本表格方式呈現，亦可採用自動化系統或軟體進行記錄。需求追溯矩陣之範例格式，請詳見附件 2。

3.2.安全軟體設計

正確的安全軟體設計可以增進軟體品質。軟體可能符合所有功能上的需求，但是並不安全。根據研究機構 IBM Systems Sciences Institute 指出，軟體在設計階段將問題排除的成本是維護階段的百分之一[13]。除了花費成本上的因素外，安全軟體設計的其他好處有：

●高存活性與高復原性。面對惡意攻擊時，進行安全設計的軟體，將可以降低攻擊成功的可能性。

●軟體品質。進行安全設計的軟體，在故意或意外的情況下發生錯誤的可能性都較低，使得軟體的可靠度較高，可靠度較高的軟體維護費用則相對降低。

●一致性與減少重新設計。未進行安全設計的軟體，若發現重大安全問題，可能因為缺乏整體安全架構而需要重新設計。

●考量邏輯上的缺陷。安全問題有時來自設計上的缺陷，而非實作上的錯誤。例如，系統設計時未考量機敏資料是否有洩漏可能，因此未加以保護，這是邏輯上的缺陷，而非實作源碼的問題。根據安全需求進行安全設計的軟體，會在早期整體考量安全機制，以避免邏輯上的缺陷。

以下將先就軟體進行設計時，對於安全性所需要考量的基本原則「安全軟體設計法則」進行介紹，後續說明如何對軟體設計進行「威脅建模」及「架

本文件之智慧財產權屬行政院資通安全辦公室所有。

構風險分析」，最後在設計階段結束前，進行「安全設計審查」，確保設計吻合需求階段提出的相關安全需求。

3.2.1. 安全軟體設計法則

●最弱環節(Weakest Link)

軟體的安全強度等同於最弱環節的安全強度。攻擊者傾向尋找整個系統中最脆弱的環節進入，而非強行突破系統防禦完善的部分。軟體設計時應從攻擊者的角度考量目前軟體架構或流程中，防護最不足甚至防護可能被繞過的情況，進行控制措施補強。

●縱深防禦(Defense in Depth)

在軟體單一環節或進入路徑採用多重的安控措施，又稱作分層防禦(Layered Defense)。採用縱深防禦的原因為，單一的安控措施被突破或失效不會造成安全危害或只造成部分的危害。以防護 SQL Injection 的例子來說明，可以對使用者輸入資料進行驗證，並於後端採用 Prepared Statements 來進行多重的防護。

●最小權限(Least Privilege)

任何使用者或程序僅在被允許的最短時間區間內，被給予必須的、最小的存取權限，以完成其作業任務。

●職責分離(Separation of Duties)

設計軟體功能時將條件設定在兩個或多個以上，且需要滿足所有的條件才能完成作業時，稱之為職責分離。軟體中進行加解密功能時，使用金鑰分持的作法，就是職責分離的一個例子。不將金鑰儲存於單一位置，將金鑰分離成多部分並分開由不同人員保管或儲存於不同位置的作法，提供了更

本文件之智慧財產權屬行政院資通安全辦公室所有。

高的安全性。另一個例子是關於審核流程，一般來說，某個重要行為若需要進行審核，通常不允許編輯送審的人員同時又擔任審核者的角色。若設計得當，職責分離可以減少單一人員或資源由於權限過大，所可能造成的安全危害。實行職責分離再加上行為的稽核記錄，可以防範內部舞弊的情況發生。

●簡化設計(Economy of Mechanism)

軟體設計越複雜，越容易產生安全漏洞。又稱作「保持簡單愚蠢」(Keep It Simple Stupid, KISS)法則。以下作法符合此法則：

－系統模組化的設計。

－可重複使用的元件與服務。

－減少不必要的功能與服務。

●安全故障(Fail Secure)

軟體發生錯誤或故障時，預設狀態為不允許存取，且維持機密性、完整性與可用性。使用者應預設不允許存取且當嘗試錯誤次數達到軟體設定水準時，不允許使用者繼續嘗試存取。軟體功能中，不允許發生錯誤時忽略錯誤繼續進行的狀況。錯誤與例外狀況時，應明確有效的進行處理，且不可直接顯示除錯用的錯誤訊息給使用者。

●完全仲裁(Complete Mediation)

每一次主體存取物件時，軟體都作為中介者，重新檢查其合法授權。通常軟體設計於登入或第 1 次存取物件時，檢查身分認證並給予授權。身分認證與授權的結果往往使用快取機制暫存憑據(Credential)作為後續存取行為時使用，不重新進行認證及檢查授權以增進效能及易用性。這樣的作法

本文件之智慧財產權屬行政院資通安全辦公室所有。

使攻擊者可採用中間人攻擊(Man-in-the-Middle, MITM)，以劫奪憑據重送的手法，欺騙該軟體偽冒成合法使用者。因此，重要的軟體功能或物件在每次存取時，應重新檢查使用者的身分認證及合法授權，以避免重送攻擊的安全危害。

●開放設計(Open Design)

安全設計不應依賴「隱藏式的安全」(Obsecurity)。「隱藏式的安全」指安全性建立於別人不知道安全性作法為何。一個開放設計的例子是，著名的加密演算法，其公式皆為公開，密文的安全性建立於金鑰的機密性，而非演算法的機密性。一個常見的反面例子，將機敏資料例如帳號密碼直接撰寫於程式源碼中。

●使用現存元件(Leveraging Existing Components)

重複使用現有的、經過安全測試或實證的元件，因此得以避免增加新元件所產生的風險。例如，軟體有資料加密需求時，採用經過公開實證的加密演算法，如 AES 或 RSA，非自行實作演算法。

●防範單點失效(Single Point of Failure)

安全設計應確保軟體不會有單點故障或錯誤造成全面停擺的可能。防範單點失效與軟體的可用性相關，為達成高可用性的目的，軟體或系統於可能發生單點失效的部分，採用分散式架構或主從架構等方式進行備用(Redundancy)設計處理。

●最少共用機制(Least Common Mechanism)

盡可能減少適用於所有使用者或角色的機制。共享的機制通常代表共享的資料，必須被很小心的設計以避免資訊流破壞安全性。另外，由於服務所

本文件之智慧財產權屬行政院資通安全辦公室所有。

有使用者或角色的機制，為了達成不同的需求，往往開放最多的功能以涵蓋全面，常常使其中某些角色取得不必要的權限。

●可接受度(Psychological Acceptability)

考量安全機制對於人員的可接受度。良好的安全機制應該盡可能的設計為容易使用、不影響原先的存取，以及盡可能作到讓使用者感受不到其存在，以增加轉換與接受的程度。

3.2.2. 軟體設計風險分析

3.2.2.1. 威脅建模(Threat Modeling)

威脅建模採用系統化的方法，以攻擊者的角度，識別可能影響軟體系統的威脅並進行評估。基於對架構與設計的了解，識別與評估威脅後，以風險高低的順序對威脅發展適當的控制措施[14][15]。

●識別安全目標與資產

安全目標是軟體高階的需求，將對軟體安全原則造成影響。包含對機密性、完整性、可用性、認證、授權、稽核、會話管理、錯誤與例外處理及組態管理產生影響的相關需求。

資訊資產是對組織有價值的項目，其損失可能損害組織完成任務的能力。須考量對組織具有商業價值、增加競爭優勢，以及為遵循法令規範須保護的項目。有關軟體所可能包含有形資產的分類，詳見表 3 所示。

表3　軟體的有形資產

分類	描述
機敏資料	任何資料其機密性、完整性、可用性遭到破壞時，將會

本文件之智慧財產權屬行政院資通安全辦公室所有。

	遭受重大不利影響者
外部實體	驅動軟體的某人或某物，且為軟體本身無法控制者
程序	處理輸入資料的工作或行為，並輸出資料者
資料流	資料於軟體或系統中移動的方法
資料儲存庫	軟體中資料暫時或持續的儲存區

資料來源：本指引整理

●建立軟體特徵檔案

透過識別軟體的屬性，建立軟體的概觀，將軟體特徵化的程序。

將執行下列任務，將結果以文件進行記錄：

-建立軟體描述與有形資產清單

描述軟體的目的與任務，並列出有形資產清單。

-建立軟體架構圖

繪製高階的軟體架構圖，描述軟體的組成及其子系統。

-識別採用的技術

包含但不限於以下項目：程式語言、函式庫、外部元件、軟體框架、應用程式容器、平台、執行環境、網路協定等。此步驟可幫助後續識別與特定技術相關的弱點。

●軟體解構

對軟體有概觀的了解後，將軟體解構成更底層的細部組成是重要的。對於軟體運作機制有完整的了解，有助於找出相關的威脅與弱點。

本文件之智慧財產權屬行政院資通安全辦公室所有。

軟體解構的方式由下列步驟組成：

-識別信任邊界(Trust Boundaries)

識別軟體中各個有形資產，其周圍的信任邊界。就每個組成元件或執行程序，考量其上游的資料流或使用者輸入是否可被信任，如不被信任，需考量以何種方式認證及授權。需要能夠確保進入特定信任邊界時，都有足夠且適當的控管，且接收的點將會驗證所有跨越信任邊界資料的正確性。

-識別資料流(Data Flow)

跨越信任邊界的資料流必須被特別的注意，因為處理的程序應該假設資料可能含有惡意的內容並驗證其合法性。通常採取將最高階的架構圖中的元件，向下解構其組成，分析其細部元件或程序之間的資料流，並反覆以上解構之動作。

-識別進入點(Entry Point)

對於攻擊者來說，軟體的進入點也就是嘗試攻擊的進入點。對於每一個進入點，應確認其具備管控措施並提供驗證與授權的檢查。

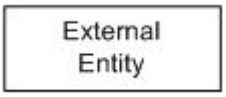
-識別特權源碼(Privileged Code)

特權源碼存取特定的資源或者進行特權行為。

解構軟體通常使用資料流程圖(Data Flow Diagram, DFD)進行輔助。資料流程圖是以圖形化表示資料流、資料儲存庫，以及資料來源與目的之間關係的圖形。資料流程圖的組成元素及其範例請詳見表 4。

本文件之智慧財產權屬行政院資通安全辦公室所有。

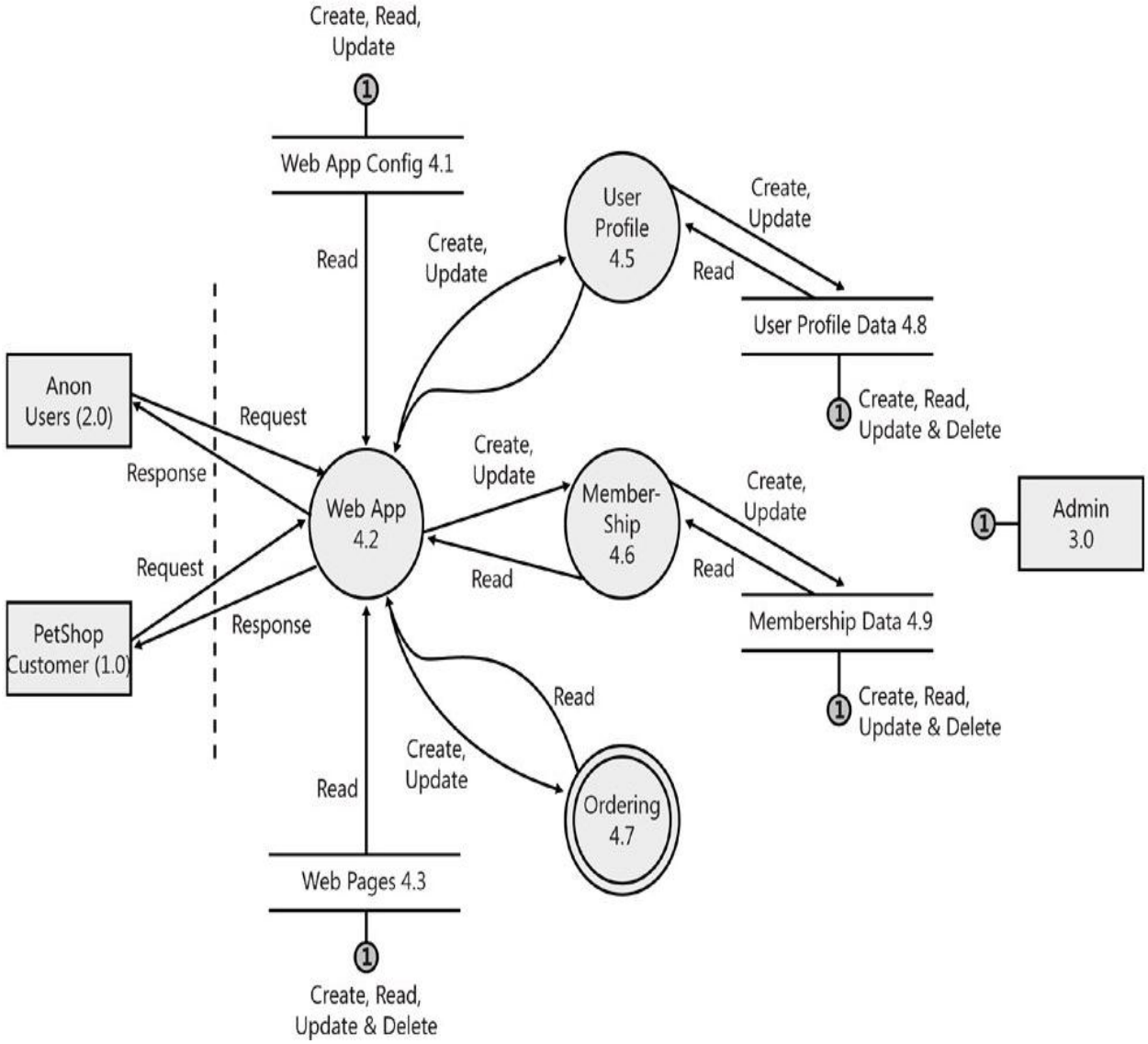
表4　資料流程圖的組成元素範例

組成元素	符號
外部實體	
程序	
資料流	
資料儲存庫	
信任邊界	

資料來源：本指引整理

有關資料流程圖之範例，詳見圖 8 所示。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[15]

圖8　Data Flow Diagram 範例

本文件之智慧財產權屬行政院資通安全辦公室所有。

●識別弱點與威脅

此步驟目的為識別影響特定資產的相關威脅。此步驟的參與人員除了軟體安全小組成員，也應包含軟體開發團隊的架構、開發、測試、作業人員在內。

以下分別說明兩種方法：

1.使用威脅類型表(例如，STRIDE)。

2.以攻擊者的角度，使用攻擊樹狀圖(Attack Tree)進行腦力激盪。

STRIDE 是一種識別威脅的簡單作法，其名稱來自 6 個威脅類型的英文字首縮寫，相關描述請詳見表 5。

表5　STRIDE

字首	代表含意	描述	相關安全特性
S	偽冒(Spoofing)	假冒為某人或某物	身分認證 (Authentication)
T	竄改(Tampering)	惡意的修改資料	完整性 (Integrity)
R	否認行為 (Repudiation)	使用者可以否認曾經進行某行為，沒有方法可以證明其行為	不可否認性 (Non-Reputation)
I	資訊洩漏 (Information Disclosure)	資訊被洩漏給不被預期可存取的個體	機密性 (Confidentiality)
D	拒絕存取服務 (Denial of Service)	對使用者拒絕服務或降低服務水準	可用性 (Availability)

本文件之智慧財產權屬行政院資通安全辦公室所有。

E	權限提升 (Elevation of privilege)	未正常授權取得權限能力	授權 (Authorization)

資料來源：本指引整理

基本上，所有 DFD 中的組成元素都是可能的攻擊目標，但根據其性質，可能產生的威脅類型並不相同，以下列出不同的 DFD 元素所可能發生、相對應的威脅類型，詳見表 6。

表6　DFD 元素對應的威脅類型

威脅類型 組成元素	偽冒	竄改	否認行為	資訊洩漏	拒絕存取服務	權限提升
外部實體	V		V			
程序	V	V	V	V	V	V
資料流		V		V	V	
資料儲存庫		V	V	V	V	

資料來源：本指引整理

針對資料流程圖中，所有的外部實體、程序、資料流及資料儲存庫，根據其可能發生的威脅類型，個別考量有哪些可能的威脅，並進行文件記錄。有關威脅描述的簡單範例詳見表 7。

表7　威脅描述範例

威脅類型 資訊資產	S	T	R	I	D	E	威脅描述

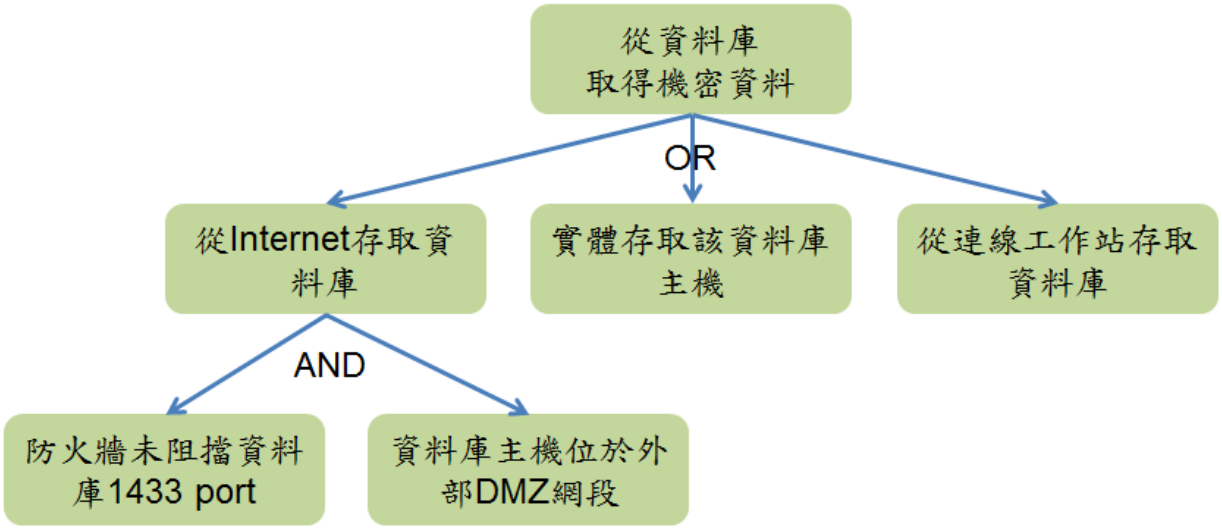
本文件之智慧財產權屬行政院資通安全辦公室所有。

威脅類型 資訊資產	S	T	R	I	D	E	威脅描述
外部實體 A	V						▪偽冒(S)：未對來源進行認證
程序 B					V	V	▪拒絕存取服務(D)：同時間可能有大量存取 ▪權限提升(E)：該程序使用過高系統權限
資料流 C		V		V			▪竄改(T)：未對資料進行完整性驗證 ▪資訊洩漏(I)：未對資料進行加密保護
資料儲存庫 D			V				▪否認行為(R)：未進行存取記錄

資料來源：本指引整理

攻擊樹狀圖(Attack Tree)是將攻擊者的目標放置於樹狀結構的根節點(Root Node)，並就此目標的達成手段開始進行分支動作。一個實際的攻擊樹狀圖範例詳見圖 9。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：本指引整理

圖9　攻擊樹狀圖範例

●以文件記錄威脅建模結果

將威脅建模的結果文件化的工作具有相當程度的重要性。在軟體發展流程中，軟體的設計可能反覆的修改，因此威脅建模的工作也是反覆的。由於必須確認最終威脅模型中識別的相關風險，其控制措施被正確的實作，詳細的文件紀錄是必須的。

●將威脅分級並排定優先次序

在此階段前，威脅建模結果應已產出針對特定軟體的威脅列表。接下來將針對各個威脅的風險程度進行評分及排序，使組織可以先處理較高風險的威脅，並依序處理其他的威脅。實務上處理所有識別的威脅可能是不符合經濟效益的作法，由於某些威脅的發生率較低或者其造成的影響很小，通常組織可能會決定忽略某些威脅，接受其可能的風險。計算軟體威脅風險的方式採用以下算式[16]：

本文件之智慧財產權屬行政院資通安全辦公室所有。

風險(Risk) = 發生可能性(Probability of Occurrence) X 商業影響(Business Impact)

此公式指出，單一威脅的風險等於威脅發生的可能性乘以造成的影響。為了應用此公式在威脅建模上，將採用 DREAD 模型來進行評估。

DREAD 名稱來自 5 個風險相關的英文字首縮寫，分別代表以下含意：

- 潛在傷害性(Damage Potential)：若此威脅發生，可能之潛在傷害。
- 可重製性(Reproducibility) ：重新產生針對此一威脅的攻擊的困難程度。
- 可刺探性(Exploitability) ：發動攻擊的困難程度。
- 影響使用者數(Affected Users)：受影響的使用者人數。
- 可發現性(Discoverability)：發現弱點的困難程度。

其中，可重製性、可刺探性、可發現性 3 者歸類在「發生可能性」，可能傷害、影響使用者數歸類在「商業影響」。

因此，應用 DREAD 模型後，可以將原先的風險公式調整為以下：

風險(Risk) = (可重製性+可刺探性+可發現性) X (可能傷害+影響人數)

組織應自行訂定威脅評分表進行 DREAD 五個指標的量化,其範例詳見表 8。

表8 威脅評分表範例

英文字首	項目	高(3 分)	中(2 分)	低(1 分)
D	可能傷害	攻擊者取得完整的機敏資料;取得	洩漏部分機敏資	洩漏系統細節

本文件之智慧財產權屬行政院資通安全辦公室所有。

英文字首	項目	高(3 分)	中(2 分)	低(1 分)
	Damage Potential	系統最高權限	料	資訊
R	可重製性 Reproducibility	攻擊可以不停重製且不需要暫停時間	攻擊可以重製,但需要暫停時間	攻擊難以重製，或需要對弱點有深入知識才能重製
E	可刺探性 Exploitability	不具有相關技能者也可以發動攻擊	具有相關技能者可以發動攻擊	需要對弱點有深入知識才能發動攻擊
A	影響使用者數 Affected Users	所有的使用者	部分使用者	極低比例的使用者
Di	可發現性 Discoverability	公開資訊已揭露攻擊手法及漏洞	已被找到相關的資訊,但漏洞尚未被揭露	極少用到的功能且使用者難以找出漏洞

資料來源：本指引整理

對應評分表後，單一威脅的風險值可以被計算出來，組織應表列所有威脅的風險值並進行高低排序，後續依照排序作為實施控制措施的優先次序。軟體的各項威脅依照風險值排序後的結果，其範例詳見表 9。

表9　威脅對應評分表範例

威脅項目	資產項目	發生可能性			影響		風險值
		可重製性	可刺探性	可發現性	可能傷害	影響人數	
未對來源	外部實體 A	2	2	2	2	3	30

本文件之智慧財產權屬行政院資通安全辦公室所有。

威脅項目	資產項目	發生可能性			影響		風險值
		可重製性	可刺探性	可發現性	可能傷害	影響人數	
進行認證							
未對資料進行加密保護	資料流 C	2	1	1	3	3	24
拒絕存取	程序 B	1	1	1	3	3	18
未對資料進行完整性驗證	資料流 C	2	1	1	1	3	16
使用過高系統權限	程序 B	1	2	2	1	2	15
未進行存取記錄	資料儲存庫 D	2	1	1	1	2	12

資料來源：本指引整理

●發展降低風險之控制措施

在此階段前，威脅建模已協助找出軟體可能的威脅並進行評分排序。本階段需針對各項威脅，以其優先次序進行控制措施的識別動作。

一項控制措施若無法將單一威脅的風險降低到可以接受的水準內，則必須採用多項控制措施。記錄威脅與相對應控制措施之範例，請詳見表 10。

表10 威脅與相對應控制措施範例

威脅項目	資產項目	風險值	控制措施

本文件之智慧財產權屬行政院資通安全辦公室所有。

威脅項目	資產項目	風險值	控制措施
未對來源進行認證	外部實體 A	30	▪採用帳號密碼登入認證 ▪限制來源 IP
未對資料進行加密保護	資料流 C	24	▪進行 SSL 加密
拒絕存取	程序 B	18	▪使用分散式系統架構
未對資料進行完整性驗證	資料流 C	16	▪進行資料訊息摘要
使用過高系統權限	程序 B	15	▪改採一般使用者權限執行
未進行存取記錄	資料儲存庫 D	12	▪對使用者行為進行記錄

資料來源：本指引整理

3.2.2.2. 架構風險分析(Architecture Risk Analysis,ARA)

「架構風險分析」是 Cigital, Inc 所提出的，在軟體設計及測試階段進行的風險分析流程[17]。架構風險分析可以分成 3 個主要的子流程：

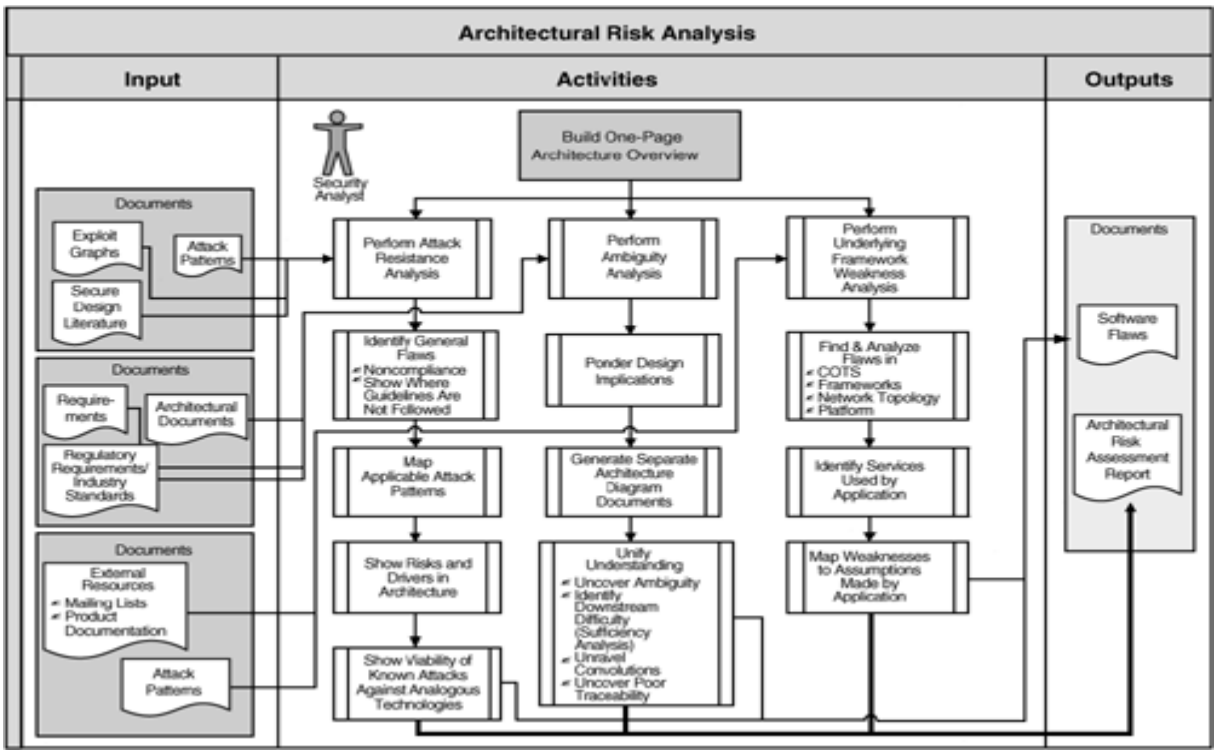
●抗攻擊能力分析(Attack Resistance Analysis)。

●模糊分析(Ambiguity Analysis)。

●底層框架弱點分析(Underlying Framework Weakness Analysis)。

「架構風險分析」整體的活動流程、輸入與產出，詳見圖 10：

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[17]

圖10 架構風險分析活動流程示意圖

3.2.2.2.1. 抗攻擊能力分析(Attack Resistance Analysis)

抗攻擊能力分析是以清單方式捕捉可能威脅的方法，此方式透過分析應用相關資訊，例如已知攻擊、攻擊模式及弱點的過程中，考量系統如何抵擋這些已知的手段，步驟介紹如下：

- 使用安全設計法則及安全特性清單(例如 STRIDE)識別一般的軟體弱點。對於過往曾經發生相關威脅的了解將有助於此一活動。
- 將弱點對應攻擊模式。使用攻擊模式清單或自行發展的誤用模型。
- 發展軟體或系統的概觀架構圖，根據清單來識別架構中的風險。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●使用刺探圖(exploit graphs)了解並展示這些已知的攻擊的可行性。

抗攻擊能力分析適合用來找出目前軟體或系統中，是否具有已知的弱點，較不適用於找出未知的問題。

抗攻擊能力分析通常可找出以下領域的安全問題：

●不安全的令牌產生與管理。

●不正確的密碼學應用。

●容易被顛覆(Subverted)的安全防護元件。

●信任或權限問題。

本節詳細作法，請參考「安全軟體設計指引」。

3.2.2.2.2. 模糊分析(Ambiguity Analysis)

模糊分析是用來發現新威脅的分析活動。這個分析需要對於軟體本身具有相當程度了解的兩位(或以上)人員。將人員分成兩個團隊分開進行相同的安全相關分析活動，然後在兩個團隊分別完成後，合併兩個團隊進行分析結果的差異分析。這樣的活動可幫助找出軟體或系統中模糊的、不一致的地方，或說是兩個團隊認知不同的安全機制作法或設計，而這些不清楚、不一致的地方正是需要進一步分析，可能找出問題之處。分析活動包含：

●信任建模：識別資料與功能在安全政策中設定的邊界。

●敏感資料建模：識別資料在軟體中信任與隱私的問題。

●威脅建模：識別要保護的功能或資料，其可能的威脅。

本節詳細作法，請參考「安全軟體設計指引」。

本文件之智慧財產權屬行政院資通安全辦公室所有。

3.2.2.2.3. 底層框架弱點分析(Underlying Framework Weakness Analysis)

底層框架弱點分析是針對軟體依賴的底層或元件，分析其可能帶來安全性影響的活動。軟體不是僅由程式設計師自行撰寫的源碼構成的。現代的軟體通常建立在複雜的軟體框架之上，並且引用眾多的外部函式庫。有些軟體依賴中介軟體(Middleware)進行管理或提供服務(例如 Web 應用程式需透過 Web 伺服器軟體提供服務)。另外，不同種類程式語言所撰寫的軟體，在作業系統的層次之上，有不同種類的執行環境(Runtime Environment)。軟體所依賴的底層或元件，若具有安全問題，亦將對軟體本身安全性造成影響。

欲找出軟體依賴所產生的安全問題，需要考量軟體的以下依賴項目：

●函式庫及其版本。

●外購套裝或客製軟體，包含提供安全特性的套裝軟體。

●軟體框架(例如.NET、JavaEE 等)。

●軟體執行的平台或環境。

●實體環境、裝置。

●建置環境。

考量的方向包含：

●依賴項目本身是否具有已知弱點？

●依賴項目的安全性是否透過某些設定或必須關閉某些特性？

●依賴項目的安全控制措施是否足夠？

本節詳細作法，請參考「安全軟體設計指引」。

本文件之智慧財產權屬行政院資通安全辦公室所有。

3.2.3. 攻擊面分析(Attack Surface Analysis)

軟體的攻擊面是以下項目的總和：

●軟體供資料或命令進出的所有路徑。

●保護這些路徑的原始碼。

●軟體中所使用，所有有價值的資料。

●保護這些資料的原始碼。

攻擊面分析則是去找出軟體或系統中，需要被審查與測試是否有安全漏洞的部分。攻擊面分析的一個重點是，使開發人員意識到軟體中的風險區域，哪些部分是開放給攻擊者的，如何最小化的方法，以及什麼時候攻擊面會異動，從風險的角度代表什麼含義[18]。

攻擊面分析的目的是：

●減少預設會開啟的攻擊面。

●減少匿名可以存取到的攻擊面。

●減少已識別身分者可以存取到的攻擊面。

●減少主體存取攻擊面時的權限。

以下說明攻擊面分析的步驟。

3.2.3.1. 識別攻擊面

從攻擊者的角度，檢視軟體設計文件與架構，找出可能的進出點(Entry/Exit)。一般來說，軟體有以下的進出點：

●使用者輸入欄位(User Input Fields)。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●協定(Protocols)。

●介面(Interface)。

●服務(Service)。

●檔案。

●資料庫。

●執行時期參數。

●其他儲存區域。

同時也需要識別軟體會使用到的，有價值的資料。

3.2.3.2. 衡量評估攻擊面

採用以下問題進行評估：

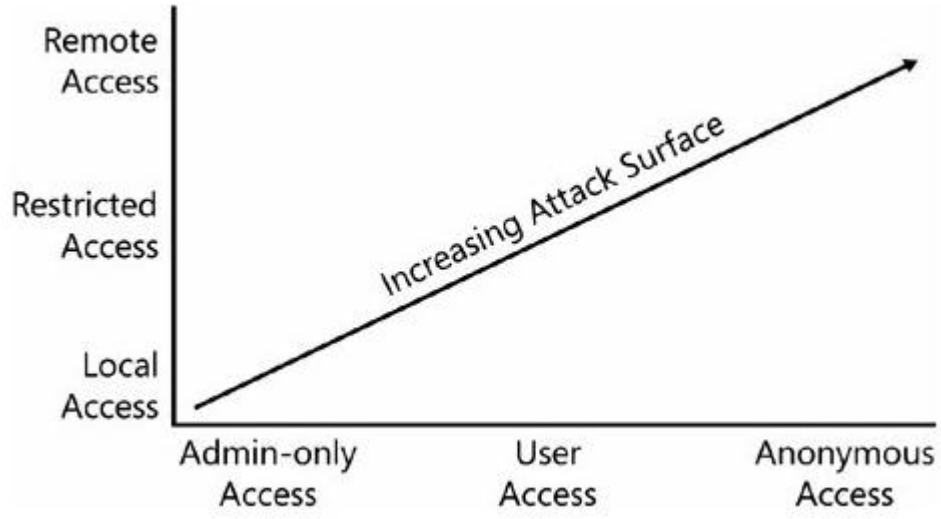
●是否大部分的使用者都需要此物件

使用 80/20 法則。是否 80%的使用者都需要此物件，如果不是，則該物件應該考慮不提供、關閉或者預設不安裝。

●誰需要存取此物件以及從什麼地方

物件允許匿名存取，跟只允許管理者存取相比，其風險較高。物件允許遠端存取，跟只允許本地端存取相比，其風險較高。因此需考量開放哪些角色及允許從何處存取。有關可存取性影響攻擊面之關係，詳見圖 11 所示。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[15]

圖11 可存取性影響攻擊面

●物件本身或存取者是否使用最小權限

檢視物件本身或者存取該物件的角色，是否只賦予該角色需要的最小權限，若不適當，則進行調整。

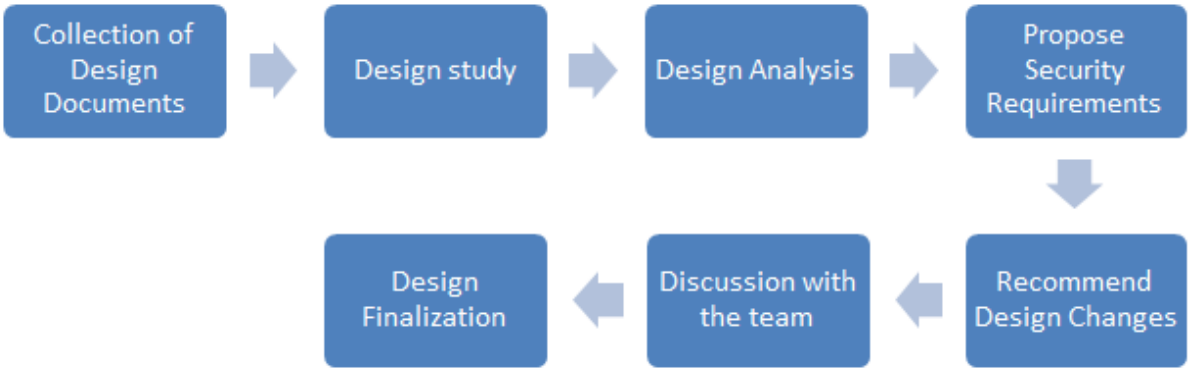
3.2.3.3. 管理攻擊面

一旦對軟體目前的攻擊面具有基本了解，後續當軟體進行異動時，可以據以評估是否導致攻擊面增加。正常來說，軟體的受攻擊面隨著時間，由於軟體功能的增加而增加。不過仍然可以透過降低使用者原先過高的權限，以及不儲存非必要的資料來減少攻擊面。

3.2.4. 安全設計審查(Secure Design Review)

本文件之智慧財產權屬行政院資通安全辦公室所有。

當軟體設計被完成，在離開設計階段進入開發階段之前，進行軟體架構與設計的安全性審查是很重要的。安全設計審查(Secure Design Review)將確保設計吻合需求階段提出的相關安全需求[19]。安全設計審查流程詳見圖 12。



資料來源：[19]

圖12 安全設計審查流程

以下簡要說明安全設計審查流程各步驟。

●收集需求與設計相關文件

收集需求與設計階段的相關系統文件，需求階段可能包含「軟體需求規格」與「需求追溯矩陣」等，設計階段可能為「軟體設計規格」等包含使用案例圖、活動圖、類別圖的文件。

●研究設計

透過與設計師或架構師討論設計文件，以分析軟體中那些地方可能是弱點而後續導致入侵。

●分析設計

本文件之智慧財產權屬行政院資通安全辦公室所有。

了解設計後下一個步驟就是分析設計中潛在的威脅。主要有兩個方向：

-缺乏安全設計

審查軟體安全需求是否都有對應的軟體安全設計。

-設計上的不安全

識別軟體設計中的「安全特點(Security Feature)」(例如，身分認證、存取控制、使用加密等)，然後檢視這些安全特點是否存在問題使得安全性不足或是完全失效。

參考 3.1.1 節的軟體安全需求類型，詳列出軟體中包含哪些安全特點的清單。一個軟體安全特點檢視結果範例，詳見表 11。後續檢視安全特點的設計，尋找可能導致安全特點失效的各種問題。

表11 軟體安全特點檢視結果範例

類型	軟體安全特點
機密性	▪傳輸過程採用 SSL 加密通道 ▪資料進行 AES 加密後儲存資料庫
完整性	▪使用資料訊息摘要，確保接受資料之完整性 ▪網站提供下載的檔案或程式，皆採用 SHA256 演算法產生 HASH 值供比對其完整性 ▪於伺服器端檢查使用者輸入合法性
可用性	▪採用分散式叢集架構確保系統可用性。 ▪資料庫定時同步至備援環境
身分認證	▪使用帳號密碼與憑證的雙重因素認證 ▪嘗試登入錯誤達一定次數後鎖定 ▪重要交易行為要求再次身分驗證，並以 Captcha 機制

本文件之智慧財產權屬行政院資通安全辦公室所有。

類型	軟體安全特點
	防堵機器人程式
授權	▪採用 RBAC 角色為主的授權機制，各角色賦予所需的最小權限，並定期檢視使用者清單 ▪重要交易行為由多人/角色授權後才得以進行 ▪重要資源存取時，再次檢查使用者權限
稽核	▪記錄使用者於軟體中的操作行為，包含特定 Session ID、存取時間、行為紀錄、存取來源 IP ▪Log 紀錄即時傳送 Syslog 伺服器作備份紀錄，以避免遭竄改或硬體損毀遺失
會話管理	▪使用者未活動一段時間即自動登出 ▪不將 Session ID 或使用者 ID 顯示於 URL
錯誤及例外管理	▪軟體發生錯誤時，使用者頁面僅顯示簡短的錯誤訊息及代碼，不包含詳細的錯誤訊息或除錯用訊息 ▪嚴重錯誤採用 Email 或簡訊通知機制
組態管理	▪軟體依賴的外部元件或軟體，不使用預設帳號密碼 ▪作業系統定期更新、關閉不必要服務、注意安全設定 ▪軟體依賴的外部元件或軟體，注意其安全通告，必要時評估更新

資料來源：本指引整理

可使用「安全軟體設計法則」來檢視軟體中主要的「安全特點」，尋找設計上的不安全處。有關「安全軟體設計法則」項目，請參考 3.2.1 節。

●修正安全需求

本文件之智慧財產權屬行政院資通安全辦公室所有。

將「安全特點」設計上的不安全處，以及缺乏的安全設計，轉化為新的安全需求或修正原本的安全需求。

●建議設計變更

將前述步驟找到的安全需求，提出相對應的設計變更建議。

●與開發團隊討論

與開發團隊討論設計變更建議項目，以確保設計變更的可行性。

●設計定型

開發團隊將設計變更納入最新版的設計規格。

本節詳細作法，請參考「安全軟體設計指引」。

3.3.安全軟體開發

安全軟體開發階段應注意避免軟體常見漏洞及實作必要控制措施，以提升軟體的安全性。程式員撰寫程式碼後，對程式碼進行安全檢查是有效發現安全缺點與未正確進行安全實作的方式。另外，組織應注重安全開發環境的管控，以減少開發活動本身產生的資安風險。

以下將分別就「常見軟體漏洞及控制措施」、「源碼安全檢查」、「安全開發環境」進行說明。「常見軟體漏洞及控制措施」中介紹一般軟體常容易發生的安全問題與對策，有關網際網路應用程式(Web Application)相關常見的跨站腳本攻擊、跨站請求偽造攻擊、命令注入攻擊，請參考「Web 應用程式安全參考指引」。

3.3.1. 常見軟體漏洞及控制措施

本文件之智慧財產權屬行政院資通安全辦公室所有。

3.3.1.1. 身分認證與會話管理不完善(Broken Authentication and Session Management)

發生原因來自脆弱的帳戶認證機制，或採取不安全的會話管理(Session Management)機制[21]。以下分別說明常見的攻擊手法及防範措施。

3.3.1.1.1. 暴力破解/字典檔攻擊(Brute Force / Dictionary Attack)

由於帳號密碼為最常見的認證方式，此類攻擊多數聚焦於猜測密碼。Brute Force Attack 為亂數組合猜測。Dictionary Attack 為組合有意義的單字進行猜測。防範此類攻擊之建議措施如下：

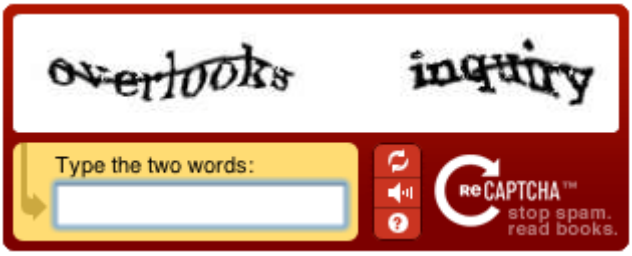
●帳戶鎖定(Account Lockout)機制

軟體的登入機制需設置單一帳戶登入錯誤次數達到限制次數時，鎖定該帳戶禁止登入之功能。考量禁止登入時間長度、是否禁止該 IP 來源以及記錄上次成功登入時間。

●採用「全自動區分電腦和人類的圖靈測試」

「全自動區分電腦和人類的圖靈測試」(Completely Automated Public Turing test to tell Computers and Humans Apart, CAPTCHA)，中文俗稱驗證碼，是一種區分使用者是電腦和人的全自動程式。由於此類攻擊通常由程式自動化進行大量重複嘗試的動作，因此 CAPTCHA 可以發揮防範效果。一個 CAPTCHA 的應用範例詳見圖 13。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[20]

圖13 CAPTCHA

●多因素認證(Multi-factor)

認證的方式可以概略區分為以下類型：

－以「使用者才知道的資訊」(Something You Know)作認證。例如，密碼、事先設定的問題與答案。

－以「使用者才擁有的資訊」(Something You Have)作認證。例如，令牌、憑證。

－以「可識別使用者本身的特徵」(Something You Are)作認證。例如，指紋、虹膜辨識。

多因素指的是認證機制採取兩個類型以上的方式進行認證。

●定期更換密碼

軟體限制使用者需定期更換密碼，新密碼不可與舊密碼重複，並且設定密碼須符合複雜度(例如，長度限制、具備英文大小寫及特殊字元等) 。

3.3.1.1.2. 密碼偷竊或密碼以明文寫於文字檔中

本文件之智慧財產權屬行政院資通安全辦公室所有。

發生原因為利用其他弱點取得資料庫或儲存媒體中的明文密碼或特權密碼直接寫於檔案中或程式碼中。防範此類攻擊之建議措施如下：

●密碼經雜湊函數(HASH)處理

密碼採用雜湊函數進行處理後才儲存檔案或媒體。使用例如 MD5、SHA1、SHA-256、SHA-512、SHA-3-512 等雜湊演算法。密碼應添加亂數資料(Salt)後進行雜湊函數處理，具備較高安全性，否則可能遭彩虹表攻擊(Rainbow Table Attack)破解。

3.3.1.1.3. 會話劫奪(Session Hijacking)

HTTP 為無狀態協定，但大部分 WEB 應用程式需要維持狀態，因此使用 Session ID 作為識別。攻擊者等待認證完成後，使用者已取得身分認證的 Session ID，再搶奪這個 Session ID，偽冒成目標。防範此類攻擊之建議措施如下：

●使用者未活動一段時間應進行登出(Idle Session Timeout)。

●使用 HTTPS 協定。

●不將 Session ID 或使用者 ID 顯示於 URL。

3.3.1.1.4. 置換會話 ID(Session Fixation)

攻擊者先行登入，取得一組 Session ID。後續置換使用者 Session ID 為攻擊者的該組 Session ID，使用者成功登入後，攻擊者的 Session ID 即被賦予權限。防範此類攻擊之建議措施如下：

●使用者登入後改變 Session ID 值。

●使用者未活動一段時間應進行登出(Idle Session Timeout)。

本文件之智慧財產權屬行政院資通安全辦公室所有。

3.3.1.2. 敏感資料外洩(Sensitive Data Exposure)

最常見的情況是未加密敏感資料，或者使用加密時，脆弱的金鑰管理機制，較弱的演算法及不足的金鑰長度。對於軟體中已經被識別的機敏資料，於實作階段應考量於傳輸時、儲存時、備份後採用加密作法。使用公開、國際機構驗證且未遭破解的演算法，不使用自行創造的加密方式。盡可能使用該演算法支援的最大金鑰長度。應對金鑰進行適當保護，金鑰應做定期更換。

3.3.1.3. 功能層級存取控制不正確(Missing Function Level Access Control)

通常發生原因是未正確對系統功能做存取控制，導致未經授權存取行為。常見種類: 1. 設定錯誤 2.功能未正確包含檢查代碼。

考量可能的問題點：

●UI 介面是否呈現連結或參數指向未授權的功能?

●是否採用 Server 端的驗證授權機制? 此機制是否運作正常?

●Server 端是否僅採用使用者提供的資訊作為驗證的依據?

防範此問題需驗證所有的系統功能。系統應包含具有一致全面性，強制適用於全系統的授權及存取控制機制。也因此，可以查驗是否有功能未使用此一機制。此一機制需預設禁止任何存取行為，根據使用者的功能存取要求，明確的判斷該使用者是否具有該權限。如果為高風險或重要的交易行為，避免使用暫存機制，應即時判斷其權限。如果此一功能為工作流程中的某一階段，與前後流程有關聯性，應判斷其狀態是否正確。

大部分的 WEB 應用程式不會向使用者顯示「未授權功能」的連結或按鈕，但這種展示層面的防範(假設使用者不知道怎麼連結到該功能)並不是合格

本文件之智慧財產權屬行政院資通安全辦公室所有。

的保護措施。必須同時採用 Server 端的驗證機制確保「真正」無法存取未授權功能。

3.3.1.4. 物件參考不安全(Insecure Direct Object References)

發生原因為軟體開發者暴露了參考物件的實作方式或邏輯,例如參考檔案、目錄名稱、資料庫紀錄等，卻未加以進行存取限制。

許多應用程式使用實際的名稱或非亂數的 Key 來作為物件的參數，但是若沒有每次驗證使用者是否具有該物件的權限，則具有此一弱點。例如，使用 http://example.com/app/accountInfo?id=123，其中 123 代表某筆紀錄，若使用者修改該數字可以參考到另一筆紀錄，同時又未加以檢查並限制存取另一筆紀錄，則發生此問題。防範之建議措施為使用「亂數化後的參數」進行參考以及「每次檢查存取權限」。

OWASP ESAPI[22]提供之 RandomAccessReferenceMap，可協助簡化達成「亂數化後的參數」此一目的，詳見圖 14。

本文件之智慧財產權屬行政院資通安全辦公室所有。

```
RandomAccessReferenceMap accessMap = new RandomAccessReferenceMap();
String indirectRef = (String)accessMap.addDirectReference("1");
System.out.println("indirectRef:"+indirectRef);

//假設使用者正確輸入
String userRequestObjId = indirectRef;
//實際上您應該取得使用者自行輸入值，例如
// String userRequestObjId = args[0];
// or
// String userRequestObjId = request.getParameter( "id" );

try {
    String directRef = (String) accessMap.getDirectReference(userRequestObjId);
    System.out.println("directRef:"+directRef);
} catch (AccessControlException e) {
    //妥善處理錯誤
    return;
}
```

資料來源：本指引整理

圖14 防範物件參考不安全之範例

以圖 14 範例來說，原先的 ID 為 1，經轉換後的亂數值 indirectRef 可能為 Stkc9C，實際上軟體應顯示經轉換後的亂數值。後續要取得原先 ID，只要把亂數值作為參數，呼叫 getDirectReference 方法即可取得。

3.3.1.5. 安全設定錯誤(Security Misconfiguration)

安全設定錯誤(Security Misconfiguration)可能發生在多個層次，包含平台、網頁伺服器、應用程式伺服器、資料庫、軟體框架與客製化程式碼等。部署環境面問題，往往對安全性帶來重大影響。此部分請參考 3.5 節詳細說明。

3.3.1.6. 使用已知含有弱點元件(Using Known Vulnerable Components)

發生原因可能為系統使用的外部元件或函式庫並非最新的版本(Stable Release)，而該元件或函式庫具有弱點，且攻擊者可從外部偵測到系統使用該元件。

本文件之智慧財產權屬行政院資通安全辦公室所有。

大部分的第三方元件，對於漏洞修補的作法是在下一個版本進行修復，而非對現在版本進行更新。因此定期檢視，評量是否升級第三方元件相當重要。軟體專案應設置流程，進行下列步驟：

- 識別所有使用到的第三方元件及其版本，包含依賴的函式庫。
- 定期檢視這些元件的公開安全資訊，包含網站、郵件列表，安全通告等，以確保接收到第三方元件的安全訊息。
- 建立安全政策指導第三方元件的使用原則，例如，需要了解其安全性(進行安全分析)、通過安全測試等。
- 若評估後可行，考慮自行封裝第三方元件，禁止使用軟體未用到的功能或安全性分析後有安全疑慮的功能。

3.3.1.7. 未驗證進行重導和導向(Invalidated Redirects and Forwards)

發生原因為軟體具備將使用者重導致其他頁面或網站的功能，此一功能接受 URL 參數時，沒有驗證的機制。此一缺失使得攻擊者可以指定導向的目標至惡意的網頁或發出特定請求存取使用者的資源。防範之建議作法如下：

- 盡可能避免於使用者輸入欄位、URL 或網頁內提供可以指定重導頁面的功能。
- 若有必要使用，則該參數不應提供使用者輸入明確的目標，而是採用間接參考的作法。例如，http://example.com/redir.jsp?url=Stkc9C，於軟體內部邏輯再行對應 Stkc9C 等於某一固定網頁 URL。
- 確保每次重導或導向的行為，導向目標皆為「合法」且「使用者具備導向該網頁之授權」。

本文件之智慧財產權屬行政院資通安全辦公室所有。

3.3.2. 源碼安全檢查(Secure Code Review)

源碼安全檢查是對軟體的源碼進行稽核的流程，以驗證正確的安全控制措施已被實作、如預期般地被實作且在正確的地方被呼叫使用[15]。源碼安全檢查是確保軟體以自我防禦的方式進行開發，軟體開發者遵循組織訂定的源碼撰寫安全規範，未使用不安全的函式，採用安全的實作方法，以避免常見的安全弱點。另外，以人工進行源碼安全檢查的方式，可以找出軟體中除錯用源碼、維護用後門與測試帳號等安全危害。

發展並採用「查檢表」是源碼安全檢查的實務作法，可以確保源碼安全檢查的涵蓋面以及具有一致性的標準。建議應包含的範圍如下[23]：

- 資料驗證(Data Validation)。
- 身分認證(Authentication)。
- 會話管理(Session Management)。
- 授權(Authorization)。
- 密碼學(Cryptography)。
- 錯誤處理(Error Handling)。
- 稽核日誌(Logging)。
- 安全組態(Security Configuration)。
- 網路架構(Network Architecture)。

有關「源碼安全查檢表」之範例，請詳見附件 3。

3.3.3. 安全開發環境

本文件之智慧財產權屬行政院資通安全辦公室所有。

組織應在整個軟體發展流程中，建立與適當保護「安全軟體開發環境」。「安全軟體開發環境」包含軟體建置與整合活動中，相關的人、程序與技術。考量以下項目：

●軟體或系統處理、儲存、傳輸資料的敏感性。

在開發或測試環境中存取使用機敏資料，需有保護措施及管制程序。

●適用的內部及外部安全需求(參考 2.4.2 節與 3.1 節)。

●組織支援軟體開發的安全控制措施

包含管理面、實體與技術的控制措施。

●軟體開發工作環境中人員的可信賴度。

人員進用前，對其進行背景查核。

●軟體發展的委外程度(參考 3.6 節)。

●區分軟體發展環境

開發、測試以及正式作業環境應作區分，以避免資料混用以及源碼版本的問題。

●控制開發環境的存取

限制對程式碼、高權限的工具程式(例如後臺管理工具)以及重要資訊的存取。

●監控軟體發展環境及其中源碼的異動

開發、測試以及正式作業環境中，程式碼版本的變動應受到控管，並記錄其異動。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●備份另行儲存在安全地點

重要資料或軟體系統應定期備份，另行儲存在非正式作業環境外的地點。

●控制不同環境間資料的複製移動

開發與測試環境應避免使用正式環境的資料，並建立程序管制不同環境間資料移動。

3.4.安全軟體測試

安全軟體測試階段是整體安全軟體發展流程中，不可或缺的階段。軟體安全是軟體品質的一部分，軟體測試之目標即為提升軟體品質，因此常參考到軟體品質確保(Quality Assurance, QA)的觀念。軟體測試包含功能性、使用者介面、效能、安全、設定及安裝等，其中安全測試著重於找出軟體中安全性相關問題。

安全測試包含兩個面向，一個是「安全性測試」，另一個是針對「安全需求驗證」，因此需區分兩者的不同。「安全性測試」是以攻擊者的角度去確認軟體承受攻擊的能力，著重在找出軟體的安全錯誤(Bug)與缺陷(Flaw)。「安全需求驗證」則是測試軟體中安全相關功能(例如，身分驗證機制、存取控制機制)是否符合需求並運作正確。

軟體「安全性測試」一般可分為「靜態分析」與「動態分析」。「靜態分析」可以在不執行軟體的狀況下，針對軟體的內容進行分析，比對已知弱點模式，找出可能的缺陷或錯誤。「動態分析」則是將軟體在實際的環境中執行起來並進行分析的活動。實務上，靜態分析通常採用「源碼檢測」，動態分析通常採用外部的「滲透測試」或「漏洞掃描」進行，而漏洞掃描

本文件之智慧財產權屬行政院資通安全辦公室所有。

即為滲透測試的階段步驟之一。以下分別就「源碼檢測」、「滲透測試」及「安全需求驗證」進行說明。

3.4.1. 源碼檢測(Source Code Analysis)

源碼檢測可分為人工檢測與採用自動化工具進行檢測。人工檢測由於檢測效率不佳、人員須具備深入的專業知識、需耗費大量人力時間等限制，加上近來源碼檢測自動化工具的發展，可對軟體專案進行全面性的檢查，快速產出軟體弱點並進行分級，針對弱點提供修補建議，且確保檢測品質的一致性等優點，目前源碼檢測活動常採用先以自動化工具進行，後續再加以人工判斷的方式。因此，了解組織本身的軟體特性與需求，選擇合適的自動化工具成為此活動的重點。以下針對選擇源碼檢測工具的規範要點(Criteria)進行說明[24][25]。

3.4.1.1. 基礎要求

- 在源碼中識別各類型軟體安全弱點。
- 回報識別出的軟體安全弱點，屬於何種類型與位於何處。
- 不可有過高的誤判率，組織應自行設定可接受的標準。

3.4.1.2. 部署支援性

- 部署模式

  採用套裝軟體安裝於個人電腦環境，或是採用軟體即服務(Software as a Service, SaaS)方式遠端提供服務。

- 安裝支援

本文件之智慧財產權屬行政院資通安全辦公室所有。

源碼檢測工具應提供安裝手冊與操作手冊。若為 SaaS 方式則應提供下列資訊：

-使用服務的相關指導。

-源碼送至該服務後估計的檢測時間。

-保護源碼安全及報告機密性的措施。

●部署架構

應提供下列資訊：

-部署架構採用伺服器端或客戶端模式。將影響是否需要特殊的權限或需要採買其他硬體設備。

-同時間可平行檢測的最大能力及是否具備擴充能力。

●執行環境

採用何種環境進行分析，使用編譯後的二進位檔或編譯前的源碼作為檢測基礎。使用編譯後二進位檔的方式，需要使用軟體依賴的外部函式庫，採用編譯前源碼方式則不需要。

3.4.1.3. 技術支援性

●程式語言

支援的程式語言種類及版本，組織應確保能夠符合本身使用的所有程式語言。

●開發環境

本文件之智慧財產權屬行政院資通安全辦公室所有。

若軟體的開發採用軟體框架或外部函式庫,則當軟體框架/外部函式庫產生安全問題，該軟體便具有該問題。因此，具備識別軟體框架、外部函式庫的能力相當重要，包含以下項目：

- 了解軟體框架如何運作，以及軟體如何與其互動、資料傳輸的能力。
- 識別軟體以不安全的方式使用框架的能力。
- 識別軟體框架中已知安全問題的能力。

●設定檔

部分安全弱點來自於不安全的設定。因此，應了解工具對於設定檔類型的支援程度，以及找出不安全設定的能力。

3.4.1.4. 檢測、命令與控制的支援性

此部分的支援性將影響使用者設定、客製化、將工具整合至組織軟體開發流程內的能力，另外也影響找出問題及修復問題的效率。

●可檢測的弱點類型

工具需能夠了解、檢測並回報源碼中的常見弱點，請詳見以下所列相關類型：

- 誤用 API(API Abuse)。
- 設定不正確(Application Misconfiguration)。
- 未關閉密碼自動完成(Auto-complete Not Disabled on Password Parameters)。
- 緩衝區溢位(Buffer Overflow)。

本文件之智慧財產權屬行政院資通安全辦公室所有。

- 命令注入(Command Injection)。
- 憑據/會話 ID 預測(Credential/Session Prediction)。
- 跨站腳本攻擊(Cross-site Scripting)。
- 拒絕存取服務(Denial of Service)。
- 權限提升(Escalation of Privileges)。
- 不安全的密碼學應用(Insecure Cryptography)。
- 字串格式(Format String)。
- 憑據以明文寫於源碼中(Hardcoded Credentials)。
- HTTP 回應分割(HTTP Response Splitting)。
- 不正確的輸入處理(Improper Input Handling)。
- 不正確的輸出編碼(Improper Output Encoding)。
- 資訊洩漏(Information Leakage)。
- 不安全的資料快取(Insecure Data Caching)。
- 不安全的檔案上傳(Insecure File Upload)。
- 帳戶鎖定機制不足(Insufficient Account Lockout)。
- 認證機制不足(Insufficient Authentication)。
- 授權機制不足(Insufficient Authorization)。
- 日誌機制不足(Insufficient/Insecure Logging)。
- 密碼複雜度不足(Insufficient Password Complexity Requirements)。

本文件之智慧財產權屬行政院資通安全辦公室所有。

－密碼週期要求不足(Insufficient Password History Requirements)。

－會話過期機制不足(Insufficient Session Expiration)。

－整數溢位(Integer Overflows)。

－目錄服務命令注入(LDAP Injection)。

－電子郵件命令注入(Mail Command Injection)。

－Null Byte 注入(Null Byte Injection)。

－開放重導攻擊(Open Redirect Attacks)。

－作業系統命令注入(OS Command Injection)。

－造訪路徑攻擊(Path Traversal)。

－競賽情況(Race Conditions)。

－遠端檔案攻擊(Remote File Inclusion)。

－置換會話(Session Fixation)。

－SQL 命令注入(SQL Injection)。

－誤用 URL 導向(URL Redirection Abuse)。

－XML 命令注入(XML Injection Attacks)。

－XML 路徑注入(XPATH Injection)。

●檢測設定

包含以下項目：

－檢測排程能力。

本文件之智慧財產權屬行政院資通安全辦公室所有。

- 檢視執行排程即時狀態能力。

- 將設定儲存為模板並重複使用的能力。

- 同時間執行多排程的能力。

- 支援多使用者的能力。

- 支援漸進式掃描的能力。

●客製化能力

- 增加/刪除/修改檢測樣式。靜態源碼分析工具常有誤判的問題，因此修改檢測樣式的功能可以幫助消除誤判，或將該筆誤判標示為安全。

- 套裝規則。提供套裝規則並可以選擇其中部分來執行檢測。

- 可選擇的檢測。具備針對特定問題重新掃描檢測的能力。

- 單元測試。具備單元測試能力以驗證新的規則。

- 教育訓練。撰寫新的規則是否需要額外的教育訓練。

●命令列支援

使用者是否可採用命令列方式啟動並執行檢測作業。

●整合式開發環境

工具是否支援組織使用的整合式開發環境(Integrated Development Environment, IDE)，可以從整合式開發環境中對軟體專案進行檢測。此方式將有助於開發人員於實作階段便進行檢測排除問題。

3.4.1.5. 規則更新

本文件之智慧財產權屬行政院資通安全辦公室所有。

檢測的規則是源碼檢測工具得以識別安全弱點的原因。因此對於規則的更新必須加以考量：

●更新的頻率

採用週期性、根據客戶要求或者其他方式。

●規則的有效性

說明如何更新規則以應付不斷演變的最新威脅。

3.4.1.6. 報告產生

檢測工具的報告是對於利害關係人最視覺化的功能呈現。工具應能夠提供多種不同的報告呈現方式以應付不同目標。例如，給開發者的報告需要較多的詳細技術資訊以協助修補弱點，給高階主管的報告則須著重於高階的摘要及相對應的風險。對以下項目進行考量：

●報告呈現方式的支援

提供「執行摘要」、「弱點所屬類型」、「弱點發現位置」、根據單一問題提出的「解決方案」、「範例源碼」。

●報告格式支援

列舉支援的報告產生格式。

●報告客製化能力

將稽核者發現註記在報告中的能力、將誤判註記並從報告中移除的能力，以及修改報告樣板增加組織標誌、標頭文字等資訊的能力。

3.4.1.7. 組織整合支援性

本文件之智慧財產權屬行政院資通安全辦公室所有。

採用源碼檢測工具時，需要考量與組織現有系統進行整合的支援度：

●整合問題管理系統(Issues Tracking System)

源碼檢測工具整合問題管理系統將有助於軟體專案後續控管安全問題的追蹤處理。

●整合風險管理系統(Risk Management System)

考量檢測結果如何影響及如何整合至組織的風險管理系統。

●聚合軟體專案的能力

將組織內採用相同程式語言或相同技術、軟體框架的專案，以採取聚合的方式呈現更多資訊，例如同類的軟體專案是否發現比其他類型更多問題?或者問題集中於某些弱點類型？

●軟體授權方式

不同授權方式對於組織總體成本的利弊。

3.4.2. 滲透測試

滲透測試是由一群具有專業與豐富的「網路及系統資訊安全」相關知識與經驗的團隊人員負責執行測試。在測試過程中，滲透測試團隊的成員，以攻擊者的思維角度，在具備高道德標準與完善的作業準備下，進行檢測作業。以掃描工具、攻擊程式及技術等輔助，對目標進行安全探測，找出資安防護機制存在的弱點或安全問題。使接受滲透測試的單位，能了解所面臨的資安問題，且後續團隊提供完整的評估報告與建議措施，解決資安問題。滲透測試執行的重點如下[26]。

●依賴人員的技術能力與經驗。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●取決於檢測人員的思維與想像力。

●需透過人工的方式與工具的輔助。

●發覺軟體或系統上的已知或未知的漏洞。

滲透測試由於實際對目標進行安全探測，與入侵行為相似，因此將面臨法律問題。為避免觸犯刑法第三十六章妨害電腦使用罪，此活動需簽署滲透測試同意書(Permission Memo)，取得相關人士的授權及明確定義測試範圍才可進行。

滲透測試的主要階段為：「需求訪談」、「資訊蒐集」、「網路與主機掃描」、「弱點利用」及「滲透測試報告撰寫」。以下分別就各階段內部活動進行說明。

3.4.2.1. 需求訪談

滲透測試需求訪談的主要目的如下：

●討論並確認測試範圍

為避免測試範圍變動，需明確定義測試範圍，什麼可以被測試，什麼不能被測試。可能對象包含特定網域、IP 區段、單一主機、特定應用程式等。另外需考量的議題包含滲透測試是針對正式營運環境或測試環境、是否採用社交工程攻擊手法、是否採用阻斷攻擊手法、是否可執行具危險性攻擊等。

●確定執行日期與時間

明確規範執行日期與時間，每日可以執行的時間、是否為 24 小時執行或離峰時段才可以執行。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●確認執行方式

採用告知相關部門方式將使測試可以更順利進行，不告知則可以測試對於發現網路攻擊的相關能力。

●制定相關規則

雙方需具備暢通的聯絡管道，當測試主機在過程中發生問題或發現遭到入侵的足跡等事件，需要能夠即時聯絡到相關人員。另外，需制定安全的資訊交換管道，例如測試過程的相關資訊提供，測試結果報告的提供等，皆需透過安全管道交換以確保機密性。

●定期進行溝通

設定時間定期討論執行進度，以確保雙方都能掌握目前的執行情況進度，並說明是否有發生重大的問題。

●撰寫執行計畫書

開始執行之前，將訪談階段的所有結論都寫進計畫書。受測單位主管與測試團隊主管需要簽署這份文件，必要時包含測試參與人員也需簽署。

3.4.2.2. 資訊蒐集

此階段已正式開始執行滲透測試。測試團隊收集任何可以從網際網路上找到的受測單位資訊，包含搜尋引擎查詢結果、受測單位公開網站可下載的資訊文件、DNS 查詢結果、Whois 查詢結果等。通過識別「人」或「系統」的行為模式，收集有關如何、何時、何地以及做甚麼事的相關資訊，後續可以發現和利用漏洞。

3.4.2.3. 網路與主機掃描

本文件之智慧財產權屬行政院資通安全辦公室所有。

透過與受測目標互動，包含採用連接埠掃描、作業系統識別、版本掃描、弱點掃描等，取得受測目標的相關資訊，例如開啟的連接、使用的作業系統、網路應用程式名稱與版本及是否存在已知的弱點。

3.4.2.4. 弱點利用

此階段為實際利用發現的弱點，嘗試取得軟體或系統的權限。利用弱點的目標為排除誤判、證明弱點可以實際造成威脅、取得權限後進行更深入的滲透測試。但實際利用弱點時，可能情況將造成應用程式無法回應或系統當機。一般弱點利用可概略分類如下：

●伺服器端：針對開啟網路連線的服務。

●使用者端：針對瀏覽器、微軟 Office、PDF 檔案、Java 等。

●權限提升：嘗試將有限的權限或一般使用者的權限提升為管理者或最高權限。

3.4.2.5. 滲透測試報告撰寫

報告格式可參考以下章節內容：

●摘要

－說明整個專案內容，包含執行日期、目標、人員。

－說明整個過程中是否有發現重大弱點及對組織可能的影響。

－列出較嚴重的弱點。

●簡介

－執行日期與時間區間。

本文件之智慧財產權屬行政院資通安全辦公室所有。

- 執行範圍。
- 參與人員。

●執行過程

- 列出每個階段的執行活動。
- 詳細說明執行過程。
- 以表格方式呈現所發現的資訊。

●執行結果

- 呈現方式以主機、弱點風險等級區分。
- 提供詳細的技術資訊。
- 弱點修補建議。

●結論

- 說明執行範圍與時間等相關資訊。
- 受測單位的整體資安現況。
- 說明執行結果，尤其高風險的弱點。
- 大方向的弱點修補方向。

3.4.3. 安全需求驗證

OWASP 組織提出的應用程式安全驗證標準：Application Security Verification Standard (ASVS)[27]，用於驗證軟體是否符合安全基準。其列舉

本文件之智慧財產權屬行政院資通安全辦公室所有。

的軟體安全需求分類及細項的安全需求驗證項目，可作為發展安全軟體測試階段參考。共區分為 13 大類如下：

●身分驗證(Authentication)。

●會話管理(Session Management)。

●存取控制(Access Control)。

●資料輸入驗證 (Input Validation)。

●儲存階段密碼學應用(Cryptography at Rest)。

●錯誤處理與日誌(Error Handling and Logging)。

●資料保護(Data Protection)。

●通訊安全(Communication Security)。

●HTTP 協定安全(HTTP Security)。

●惡意程式控制措施(Malicious Controls)。

●商業邏輯(Business Logic)。

●檔案與資源(Files and Resources)。

●行動裝置(Mobile)。

本節詳細作法，請參考「安全軟體測試指引」。

3.4.4. 測試資料保護

保護測試資料的目的在於確保軟體使用的測試資料是謹慎被選取，且進行保護與控制。應避免使用正式作業環境中包含個資或敏感資訊的資料，來

本文件之智慧財產權屬行政院資通安全辦公室所有。

作為測試的用途。使用正式作業環境資料作為測試用途時，應注意以下事項：

●用於正式作業環境中的存取控制程序，也應實施於測試環境。

●每次正式作業環境中的資訊要複製到測試環境時，單獨進行授權。

●正式作業環境的資訊用於測試環境，完成測試後應立即刪除。

●複製及使用正式作業環境資料，應被記錄以提供稽核證據。

3.5.安全軟體部署與維運

當軟體已被客戶正式接受，或已經準備好被發布安裝時，安全軟體發展流程將進入部署與維運階段。軟體的安裝部署影響到軟體上線後的整體安全性，須強化部署環境並注意正確的安全設定。維運階段則需要注重變更管理、監控量測、事故處理。

3.5.1. 強化部署環境

強化部署環境工作需考量現代的軟體通常建立在其他底層之上，並與外部軟體或系統溝通，一般來說，至少需考量以下項目：

●作業系統

- 定期 Pach 更新，重要伺服器應於測試環境測試完畢後再行更新。
- 關閉不使用的服務及 Port。
- 正確的安全設定

本文件之智慧財產權屬行政院資通安全辦公室所有。

政府組態基準(Government Configuration Baseline)目的在於規範資通訊終端設備(如：個人電腦) 的一致性安全設定(如：密碼長度、更新期限等)，以降低成為駭客入侵管道，進而引發資安事件之疑慮。

-注意安全警示(Security Alert)網站之通告。

●網頁或應用程式伺服器

-諮詢廠商關於良好的安全性設定並據以實行。

-訂閱廠商的安全通告，定期評估是否更新伺服器版本。

-應用程式部署設定檔

部署設定檔通常包含了重要的安全設定，包含 Session Timeout 時間、認證方法、錯誤處理頁面及安全限制等。

●資料庫

-不使用預設帳號密碼。

-不使用過高的管理權限連接資料庫。

-前後台(一般功能與管理功能)不使用相同帳號連接資料庫。

-定期更新資料庫的安全修補。

-盡可能採用加密連線。

-存取行為進行監控，定期稽核。

●軟體本身

-敏感資料(例如帳號密碼)不以明文方式放置於設定檔或源碼中。

本文件之智慧財產權屬行政院資通安全辦公室所有。

－移除除錯用的源碼。

－移除維護用的後門。

－移除測試帳號。

－不開啟底層非必要的服務。

－以所需的最低權限執行軟體。

當軟體的底層作業平台，包含作業系統、資料庫與中間軟體平台，產生異動變更時，軟體本身也應檢視與測試以確保對於安全性與組織作業沒有不利影響。相關程序包含：

●作業平台異動變更後，檢視應用程式，以確保控制與完整性程序沒有被破壞。

●確保有關作業平台異動被及時提早通知，以允許適當的測試與檢視在異動實施之前被進行。

●確保營運持續計畫(Business Continuity Plan, BCP)進行適當變更。

3.5.2. 變更管理

軟體於維運階段中，不可避免地將會因客戶需求或軟體具有錯誤等原因，而必須進行程式的異動更新。一般來說，軟體變更管理涉及多項流程，應將軟體安全性的影響考量在內[28]：

●識別可能的變更

－因遭遇軟體錯誤或要求新功能而產生變更的需求。

－提出變更要求。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●分析變更請求

－分析變更的技術可行性。

－分析成本與效益。

●最終評估變更

－變更管理者根據可行性、成本與效益決定是否進行變更。

●計畫變更

－分析變更的影響，包含對「軟體安全性」的影響。

－對變更進行計畫。

●實作變更

－根據計畫實作變更。

－測試變更，是否運作正常與滿足變更需求。

－更新相關文件，包含「軟體安全性」的部分。

－釋出變更。反應變更需求的版本被公開上線。

●檢視與結案

－驗證變更生效。

－執行結案程序。

3.5.3. 監控量測

維運階段進行持續性的監控，可提供組織驗證其軟體運算環境的正常運作。其目的如下：

本文件之智慧財產權屬行政院資通安全辦公室所有。

●確保軟體或系統的健康狀態。

●確保安全控制措施正常到位。

●確保符合法令法規。

●提供歷史紀錄。

日誌紀錄分析是主要的監控手段之一。日誌紀錄除了一般用來進行系統除錯、作為稽核證據符合法規需求外，可用來追蹤使用者行為及發現異常違反活動，具有相當重要性。日誌紀錄的常見問題包含完整性被質疑、缺少關鍵資訊以及系統錯誤或備份失誤導致紀錄喪失。日誌紀錄應考量以下項目：

●保留時間

－Log 應保留多久，是否有法規要求。

－相對應的儲存空間需求。

●關聯性

－不同設備會產生不同的紀錄格式，導致事件分析的困難。

●效能

－記錄所有的行為可能影響軟體效能。

－應判別系統行為的重要性，並將 Log 進行優先權分級。

除了發生程式異常錯誤外，考量以下狀況是否需進行日誌記錄：

●認證機制(Authentication)

－認證成功及失敗。

本文件之智慧財產權屬行政院資通安全辦公室所有。

－帳戶鎖定。

●資料存取(Data Access)

－異動及刪除。

－敏感資料被讀取。

－資料結構變更。

－管理者行為。

●功能錯誤(Error Conditions)

－查詢語法執行失敗。

－檔案存取錯誤。

－非預期的狀態。

－連線失敗(資料庫或其他外部系統等)。

－逾時、效能問題。

紀錄內容應考慮包含：

●使用者 ID

－唯一識別的 ID，但個資類型(身分證字號、Email)應除外。

－避免多人共用同一帳號，造成無法歸責狀況。

●時間

－系統應設定定期校時。

－應準確至微秒(Milliseconds)。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●網路位址(Network Address)。

●行為描述

－存取的資源。

－進行的方法。

●錯誤代碼

－便於搜尋及問題追查。

－避免不同錯誤使用相同代碼。

●優先權

－決定後續例外處理及通知行為。

紀錄內容不應包含：

●敏感資訊，如個資。

●政府核發的證照編號，例如身分證號、護照號碼等。

●信用卡資訊、財務資訊。

●醫療資訊。

●生物特徵資訊。

●個人即時的地理位置所在。

●個人通聯內容。

●密碼。

本文件之智慧財產權屬行政院資通安全辦公室所有。

要達成持續性監控的目的，除了日誌紀錄分析外，也需要有效的量測(Metrics)。當應用於安全領域時，量測必須被清楚的定義。量測指標有助於了解當前軟體的安全狀態並發現違反事項。良好的量測具備以下特性：

●一致性： 無論執行多少次量測，都會得到相類似結果。

●量化：以數字呈現。

●客觀性：不同人員來執行量測會得到相同結果。

●具體的背景：具體的背景使得比較相關趨勢成為可能。例如，以「一個月內發現的企圖入侵次數」，取代「共發現多少企圖入侵次數」。

維運階段中，組織可透過蒐集、分析、回報軟體安全相關資料，來促進效能與達成安全目標。採取以下步驟：

●識別要蒐集的量測指標。

●確定指標將如何使用。

●建立資料蒐集和報告策略。

●定期收集和評估指標。

3.5.4. 事故處理

相對於持續性監控活動追蹤監看可能對軟體造成損害的企圖，事故處理則是當安全事故或破壞真正發生時，所遵循的處理步驟。事故回應處理流程可以概分為以下 4 個階段[29]：

●準備階段

－建立事故處理團隊並給予適當資訊安全教育訓練。

本文件之智慧財產權屬行政院資通安全辦公室所有。

- 建立事故處理程序，定期進行演練。
- 準備事故緊急聯絡資訊清單(包含內外部成員)，以及事故處理必要資源。

●偵測與分析

- 事件回報。由人員或偵測機制通知事件的發生，或經由媒體得知相關訊息。
- 識別事故。經由日誌紀錄或其他資訊，判斷分析事件是否對組織造成影響，成為安全事故。
- 通知事故並組成團隊。若判斷為安全事故，根據程序通知適當人員進行事故處理。

●遏制、根除與回復

- 遏制。事故首先必須進行遏制動作，避免進一步的傷害以及風險。遏制的作法有：將軟體或系統關閉、網路斷線、以網路設備阻擋連線、停止讓使用者登入等。正確的遏制作法相當重要，若採取不適當的遏制作法將可能使事故的證據無法被充分收集。建議於事故處理程序定義不同事故類型所對應採取的遏制作法。
- 根除。根除是將問題實際消除的步驟。以軟體而言，最常見的情況就是進行源碼撰寫，修補相關弱點後進行更新。若問題來自軟體依賴的外部函式庫或框架，則應取得正確更新檔進行修補。根除步驟實施後，應進行測試以確保問題確實被解決。
- 回復。確保軟體或系統回到原本正常運作的狀態。需驗證重要功能是否運作正常，並監視相同的事故是否再次發生。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●事故後分析

事故後分析之目的為從事故中學習。可採用會議方式，研討以下問題：

－事故發生詳細過程與發生次數。

－是否依照程序文件進行處理，程序文件的適切程度。

－需要保留哪些事故資訊。

－需要提供哪些單位相關資訊。

－未來可採取何種預防及偵測措施，是否需要增加相關資源。

3.6.安全軟體委外開發管理

採取委外專案方式的原因有多種,通常組織著眼於其中優勢及獲得的利益。以下概述採取委外的動機：

●集中內部資源在核心的業務功能，可以減少成本及增加投資報酬率。

●獲取組織尚未擁有的，短期或高度專業化的能力。

●獲取共通、基本的服務或工具。

●為使業務可以在不同的地理區域進行作業。

●獲取軟硬體設備使得勞動生產力提高。

委外專案主要有「規劃分析」、「招標評選」與「履約管理」階段，有關各階段詳細的活動內容，並不在本指引的討論範圍內。然而，考量軟體委外時,由於委外單位無法實際參與軟體發展流程,如何顧及軟體的安全性，確保相關資安風險被有效識別分析，控制措施有效實作，是更為困難的挑

本文件之智慧財產權屬行政院資通安全辦公室所有。

戰。以下就委外專案各階段，為確保軟體安全性，所應注意的要點進行說明。

3.6.1. 規劃分析

由於委外單位難以實際參與供應商開發流程，因此事前對於軟體安全需求必須清楚識別，置入需求規格書作為供應商建置依循。

●識別軟體中的資訊資產

請參考 3.2.2.1 節，識別並整理該委外軟體所包含的資訊資產以及安全目標。

●識別威脅與風險

- 軟體處理、儲存、傳輸的機敏資料，應考量其威脅與風險，進行資料分級。請參考 3.1.2.2 節。
- 採用軟體安全問卷，識別一般軟體所會面臨的各類型安全問題。請參考 3.1.2.3 節及附件 1。
- 繪製該軟體的使用案例，包含具備哪些功能，哪些使用者角色或外部系統會與這些功能互動，完成後據以發展「誤用案例模型」。請參考 3.1.2.4 節。
- 發展「主體-物件矩陣」，列出各角色於軟體中的權限、被允許存取的物件，配合安全軟體設計法則檢視是否存在安全問題。請參考 3.1.2.5 及 3.2.1 節。

●產生安全需求

本文件之智慧財產權屬行政院資通安全辦公室所有。

- 來自法令法規或內部安全政策的需求，必須加以考量並產出相關安全需求。請參考 3.1.2.1 節。
- 資料分級結果，可據以設立安全需求。例如，風險等級高的資料，在傳輸時須加密保護。請參考 3.1.2.2 節。
- 來自「軟體安全問卷」填答結果。請參考 3.1.2.3 節及附件 1。
- 來自「誤用案例模型」及「主體-物件矩陣」找出的安全風險。請參考 3.1.2.4、3.1.2.5 節。
- 針對軟體安全需求舉行討論會議，進行腦力激盪。請參考 3.1.2.6 節。

產出的軟體安全需求，可以放入需求追溯矩陣(Requirements Traceability Matrix, RTM)進行記錄與後續追蹤。請參考 3.1.3 節。

依據「行政院所屬各機關資訊業務委外服務作業參考原則」，重要資訊專案得視需要區分顧問標、規劃標、建置標以及監督審驗標辦理[30]。該委外軟體如有重大資安風險考量，此階段活動可考慮採另行辦理。

3.6.2. 招標評選

委外專案針對供應商評選之指標眾多，且需考量不同專案特性進行選擇，不在本文討論範圍內。然而，針對軟體委外安全需求，應多加注意的供應商特點為：

- 對於同類具有資安風險系統的開發經驗。
- 廠商資安國際認證與人員資安專業證照數。
- 具有軟體安全測試工具及問題修補能力。

本文件之智慧財產權屬行政院資通安全辦公室所有。

前述 3.6.1 節,「規劃分析階段」產出的軟體安全需求,應在需求規格(Request For Proposal, RFP)中詳細完整呈現,並請供應商於計畫書中說明如何驗證軟體安全需求已實作完成並符合預期要求。

委外單位於訂定交付項目時，可考慮將安全軟體發展流程中，各階段遵照安全原則所進行的各項活動產出納入交付項目：

●安全軟體設計階段

－進行威脅建模後，產出的威脅及控制措施。請參考 3.2.2.1 節。

－安全特點檢視結果。請參考 3.2.4 節。

●安全軟體開發階段

－源碼安全查檢表之檢核結果。請參考 3.3.2 節。

●安全軟體測試階段

－源碼檢測結果報告。請參考 3.4.1 節。

－滲透測試或漏洞掃描結果報告。請參考 3.4.2 節。

●安全軟體部署與維運

－強化部署環境程序。請參考 3.5.1 節。

－變更管理程序。請參考 3.5.2 節。

－監控量測程序。請參考 3.5.3 節。

－事故處理程序。請參考 3.5.4 節。

3.6.3. 履約管理

本文件之智慧財產權屬行政院資通安全辦公室所有。

履約管理階段對於軟體委外專案最終的安全程度有重要影響。供應商在各階段產出與安全相關的交付項目，委外單位應驗證其檢核結果並要求供應商進行改善。

●安全軟體設計階段

－驗證控制措施是否正確、有效的實作完成。

－驗證安全特點檢視結果之正確性。

●安全軟體開發階段

－驗證源碼安全查檢表之正確性。

●安全軟體測試階段

－根據事前設定的標準，追蹤確認源碼檢測結果報告中的安全弱點被修正。

－根據事前設定的標準，追蹤確認滲透測試或弱點掃描結果報告中的安全弱點被修正。

●安全軟體部署與維運

－確認部署上線時，強化部署環境之程序如實被執行。

－確認供應商按照變更管理程序執行軟體維護。

－確認監控量測程序有效執行。

－確認事故處理程序有效執行。

本文件之智慧財產權屬行政院資通安全辦公室所有。

4. 結論

本指引針對國際現有與安全軟體發展生命週期相關流程與規範進行探討，進而歸納導入安全軟體發展流程之管理重點，包含策略與評量、政策與法規遵循與教育訓練。

本指引之核心係整理出安全軟體發展流程階段實務之各階段活動要項，包含需求階段的安全需求萃取與產生，安全軟體設計的法則與方法論，安全軟體開發上常見的漏洞與控制措施，與測試驗收階段的技術驗證與需求驗證，提供相關的理論與實作介紹，確保安全軟體發展實務中的可驗證之需求的可執行性。

而後針對軟體完成後之後續部署與維運，提供管理、變更、監控量測與事故處理之管理建議，確保軟體系統後續持續營運之安全性。

最後針對軟體委外開發業務，事前之規劃分析、招標評選與後續之履約管理上，提供針對本指引中各生命週期中的執行項目可參考之建議，確保委外開發業務管理上，能透過合約制定合適的安全需求的要求與交付驗收項目，確保委外履約管理上對於安全需求的驗收的可執行性。

本指引主要為提供機關於軟體自行開發或委外管理上之參考依據，期能落實全軟體生命週期之實踐，有效從源頭強化整體資訊系統之安全性，強化政府整體資安能量。

本文件之智慧財產權屬行政院資通安全辦公室所有。

5. 參考文獻

[1] 安全的系統發展生命週期(SSDLC)介紹。賴溪松。

[2] http://en.wikipedia.org/wiki/Requirements_analysis, wiki

[3] Simplified Implementation of the SDL, Microsoft

[4] https://www.owasp.org/index.php/Category:OWASP_CLASP_Project, OWASP

[5] http://www.iso.org/iso/home/store/catalogue_tc/catalogue_detail.htm?csnumber=54534,ISO

[6] http://www.iso.org/iso/catalogue_detail.htm?csnumber=44378,ISO

[7] "Security Considerations in the System Development Life Cycle", The National Institute of Standards and Technology (NIST),http://csrc.nist.gov/publications/nistpubs/800-64-Rev2/SP800-64-Revision2.pdf, NIST 800-64

[8] http://bsimm.com/online/, BSIMM

[9] https://www.owasp.org/index.php/SAMM#tab=OpenSAMM, SAMM

[10]Official ISC2 Guide to the CSSLP, ISC2

[11]An Introduction to Computer Security: NIST SP800-12

[12]https://www.cigital.com/products/elearning/, Cigital

[13]http://www.softwaremag.com/DSN/wwwswmagcom/Content/ClientAssets/ISC_WPaper_03-20-14.pdf

本文件之智慧財產權屬行政院資通安全辦公室所有。

[14]A Practical Approach to Threat Modeling, Tom Olzak

[15]The Security Development Lifecycle, Microsoft

[16]Improving Web Application Security Threats and Countermeasures,Microsoft

[17]Software Security-Build Security In, Gary McGraw

[18]https://www.owasp.org/index.php/Attack_Surface_Analysis_Cheat_Sheet,OWASP

[19]https://www.owasp.org/index.php/OWASP_Secure_Application_Design_Project#tab=Methodology_of_Design_Review, OWASP

[20]http://www.google.com/recaptcha/intro/index.html

[21]https://www.owasp.org/index.php/Top_10_2013-Top_10, OWASP

[22]https://www.owasp.org/index.php/Category:OWASP_Enterprise_Security_API, OWASP

[23]OWASP Code Review Guide, OWASP

[24]http://projects.webappsec.org/w/page/66094278/Static%20Analysis%20Technologies%20Evaluation%20Criteria

[25]http://samate.nist.gov/docs/source_code_security_analysis_spec_SP500-268_v1.1.pdf

[26]Penetration Testing，http://net.ndhu.edu.tw/~net/DL/20121115.pdf，洪光鈞

[27]https://www.owasp.org/index.php/Category:OWASP_Application_Security_Verification_Standard_Project, OWASP

本文件之智慧財產權屬行政院資通安全辦公室所有。

[28]http://en.wikipedia.org/wiki/Change_management_(engineering), wiki

[29]http://csrc.nist.gov/publications/nistpubs/800-61rev2/SP800-61rev2.pdf, NIST

[30]行政院所屬各機關資訊業務委外服務作業參考原則，行政院

本文件之智慧財產權屬行政院資通安全辦公室所有。

6. 附件

附件1 軟體安全問卷範例

附件2 需求追溯矩陣範例

附件3 源碼安全查檢表範例

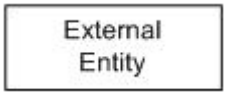
本文件之智慧財產權屬行政院資通安全辦公室所有。

附件1軟體安全問卷範例

Date:

軟體安全問卷			
I.軟體識別與所有權			
軟體識別 ID		名稱	
描述			
開發單位			
開發團隊 聯絡窗口		開發團隊窗口 聯絡資訊	
業務單位			
業務單位 聯絡窗口		業務單位窗口 聯絡資訊	
其他相關人			
高階安全需求			
採用技術			
II.軟體架構圖			

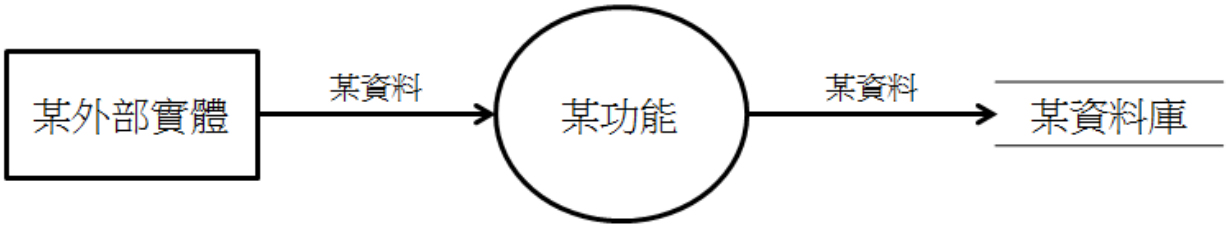
創建一個架構圖說明軟體功能與結構的組成，參考以下組成元素：

組成元素	符號
外部實體	

本文件之智慧財產權屬行政院資通安全辦公室所有。

程序	
資料流	
資料儲存庫	
信任邊界	

一個簡單的範例如下：



請在以下空白區域創建軟體架構圖。

本文件之智慧財產權屬行政院資通安全辦公室所有。

III.軟體組成	
請根據第 II 部分的軟體架構圖，整理以下軟體組成：	
主要功能描述	
定義：邏輯的流程或描述，展現該軟體的一或多個特定任務	
包含的機敏資料	
定義：任何資料若機密性、完整性、可用性發生問題時，將造成影響者	
外部實體	
定義：某人或某物將會驅動或影響軟體，但軟體本身無法控制者。例如，系統使用者、其他軟體或應用程式等	
資料儲存庫	
定義：持久化的資料儲存方式，例如檔案、資料庫等	

本文件之智慧財產權屬行政院資通安全辦公室所有。

IV.軟體安全相關問題		
分類	問題	答案 (是/否/不適用)
機密性	機敏資料傳輸時，採用加密機制	
	機敏資料儲存時，採用加密機制	
	使用公開、國際機構驗證且未遭破解的演算法	
	使用該演算法支援的最大金鑰長度	
	不使用自行創造的加密方式	
	加密金鑰具有保護機制	
	加密金鑰或憑證週期性更換	
完整性	重要資料產生 HASH 值，確保其完整性	
	重要資料傳輸過程，使用防止竄改的協定	
	提供下載的資料，產生 HASH 值供比對其完整性	
可用性	評估服務重要性，設定可用性要求	
	採用「高可用性」(High Availability) 架構或機制	
	重要資料定時同步至備援環境	
輸入驗證	採用過濾機制，以防止輸入惡意命令或資料	

本文件之智慧財產權屬行政院資通安全辦公室所有。

	驗證使用者輸入資料	
	驗證外部取得的資料	
	驗證系統參數合理性	
	於伺服器端檢查輸入資料合法性	
身分認證	除了允許匿名存取的功能外，所有功能都必須經過認證才允許存取	
	身分認證機制位於伺服端且採用集中管理機制	
	採用多重因素認證(兩種以上認證類型)	
	採用 CAPTCHA 機制於身分認證或重要交易行為，以防範自動化程式之嘗試	
	身分認證相關資訊不以明文傳輸	
	身分認證相關資訊不存於源碼中，並限制存取	
	身分認證失敗達一定次數後鎖定該帳號	
	身分認證發生錯誤時，預設不允許存取任何非公開功能	
	密碼添加亂數資料(Salt)後進行雜湊函數(HASH)處理，才加以儲存	
	密碼須符合複雜度(長度限制、具備英文大小寫及特殊字元等)	
	限制需定期更換密碼	

本文件之智慧財產權屬行政院資通安全辦公室所有。

	重要交易行為要求再次身分認證	
授權與存取控制	採用伺服端的集中管理機制檢查使用者授權	
	執行功能或存取資源前，檢查使用者授權	
	除特殊管理者權限外，其他角色或權限無法修改授權資料及存取控制列表(ACL)	
	使用者/角色賦予所需的最小權限	
	軟體程序(process)以最小的權限執行，不以系統管理員或最高權限執行	
	重要行為由多人/角色授權後才得以進行	
日誌紀錄	認證失敗、存取失敗、輸入驗證失敗、重要行為、重要資料異動、功能錯誤及管理者行為進行 Log 記錄	
	Log 紀錄考慮包含以下項目 1.識別使用者之 ID(不可為個資類型)。2.經系統校時後的時間戳記。3.執行的功能或存取的資源。4.事件類型(例如，成功或失敗)。5.事件優先權(priority)。6.事件詳細描述。7.事件代碼。8.網路位址	
	採用單一的 Log 機制，確保輸出格式的一致性	
	Log 進行適當保護及備份，避免未經授權存取	
會話管理	會話識別碼(Session ID)是隨機產生且不可預測	
	使用者的會話階段，設定在合理的時間內失效	

本文件之智慧財產權屬行政院資通安全辦公室所有。

	使用者的會話階段，使用者登出後失效	
	使用者重新登入後，會話識別碼(Session ID)會改變	
	不將會話識別碼(Session ID)或使用者 ID 顯示於使用者可以改寫處	
錯誤及例外管理	所有的功能都會進行錯誤及例外處理，並將資源正確釋放	
	軟體發生錯誤時，使用者頁面僅顯示簡短的錯誤訊息及代碼，不包含詳細的錯誤訊息或除錯用訊息	
	嚴重錯誤採用通知機制(例如電子郵件或簡訊)	
組態管理	管理者介面限制存取來源或不允許遠端存取	
	參數設定或系統設定存放處，限制存取或進行適當保護	
	依賴的外部元件或軟體，不使用預設帳號密碼	
	作業平台定期更新、關閉不必要服務、注意安全設定	
	依賴的外部元件或軟體，注意其安全漏洞通告，必要時評估更新	

資料來源：本指引整理

本文件之智慧財產權屬行政院資通安全辦公室所有。

附件2 需求追溯矩陣範例

專案名稱					專案經理		專案期程		
專案說明									
編號	功能 ID	高階需求	功能需求	狀態	設計/技術規格	實作 模組/元件	測試案例 編號	測試時間	測試 結果
1	1.1.1	認證機制進行安全防護	登入錯誤次數設定上限，超過上限時鎖定該帳號	結案	XX 系統設計規格書 P.31	實作於身分認證模組	T1.1.1	YY/MM/DD	通過
2	1.1.2	認證機制進行安全防護	使用者密碼 HASH 儲存	開發中	XX 系統設計規格書 P.32	實作於身分認證模組	T1.1.2	N/A	N/A

資料來源：本指引整理

註:狀態一般可分為開啟、已分配、開發中、已開發、測試中、測試通過、測試不通過、已上線、結案

附件3 源碼安全查檢表範例

項目	檢核結果		
資料驗證(Data Validation)	是	否	不適用
確認驗證資料合法性的機制存在	☐	☐	☐
確認所有可能被惡意使用者修改的輸入處，其輸入資料都被正確的驗證	☐	☐	☐
確認所有輸入資料的正確長度檢查存在	☐	☐	☐
確認輸入欄位、客戶端暫存、HTTP 協定標頭皆有被驗證	☐	☐	☐
確認資料的格式正確且包含的是合法的字元	☐	☐	☐
確認資料驗證機制存在於伺服器端	☐	☐	☐
檢查資料驗證是否確實發生	☐	☐	☐
檢查資料驗證機制其相關程式碼，沒有後門或惡意代碼存在	☐	☐	☐
身分認證(Authentication)	是	否	不適用
確認所有內部與外部的連線(使用者及元件)有適當而足夠的驗證機制	☐	☐	☐
確認所有非公開的功能頁面強制要求認證	☐	☐	☐
若採用 HTTP 協定，確認身分認證的憑據或其他機敏資料是透過 HTTP POST 方法傳送	☐	☐	☐

任何不在認證機制管控範圍內的功能頁面，已完成檢視評估沒有安全危害可能	□	□	□
確認身分認證的憑據不會以明文方式傳送，採用加密方式傳送	□	□	□
確認跳過身分認證的測試用、除錯用後門，沒有存在正式上線的程式碼中	□	□	□
會話管理(Session management)	是	否	不適用
檢查會話的唯一識別值(Session ID)在建立連線時，尚未身分認證前，即已經賦予使用者	□	□	□
檢查會話的唯一識別值(Session ID)的複雜度有足夠強度(採亂數方式非流水號)不會被偽冒	□	□	□
確認軟體正確的追蹤記錄 Session ID 的活動行為	□	□	□
決定當發現不合法的 Session ID 時採取的行動	□	□	□
檢查是否有不合法的 Session ID	□	□	□
具備未活動一段時間後自動登出功能	□	□	□
檢查登出功能是否正常運作，正確將 Session ID 無效化	□	□	□
授權(Authorization)	是	否	不適用
確認授權機制存在	□	□	□
確認軟體明確定義登入使用者的類型及其權限	□	□	□

確認使用者權限的賦予符合最小權限法則，僅給與該使用者所需的最小權限	□	□	□
確認授權機制運作正常、無法被繞過	□	□	□
確認每次存取要求時，權限都會被檢查	□	□	□
確認正式上線的程式碼中未包含測試用、除錯用的後門	□	□	□
密碼學(Cryptography)	是	否	不適用
確認機敏資料不以明文進行傳送(包含內、外部)	□	□	□
確認軟體採用公開、國際機構驗證且未遭破解的演算法，不使用自行創造的加密方式	□	□	□
錯誤處理(Error handling)	是	否	不適用
確認所有具有回傳值的功能呼叫，發生錯誤時都會有正確的錯誤處理，並回傳值進行確認。	□	□	□
確認例外與錯誤狀況都正確被處理。	□	□	□
確認錯誤時軟體內部詳細除錯用訊息不會回傳給使用者	□	□	□
確認錯誤時會將使用的資源正確的釋放	□	□	□
稽核日誌(Logging、Auditing)	是	否	不適用
確認錯誤發生時，記錄的資訊中不會有機敏資料	□	□	□
確認記錄的片段大小在定義的長度限制內，且日誌記錄	□	□	□

的機制會強制執行			
確認軟體會記錄客戶端使用者的行為，例如資料的操作/增刪查改	□	□	□
確認成功與不成功的身分認證都會被記錄	□	□	□
確認應用程式本身發生錯誤時都會被記錄	□	□	□

資料來源：本指引整理